

云原生训练营

模块十二： 基于 Istio 的高级流量管理

孟凡杰

前 eBay 资深架构师

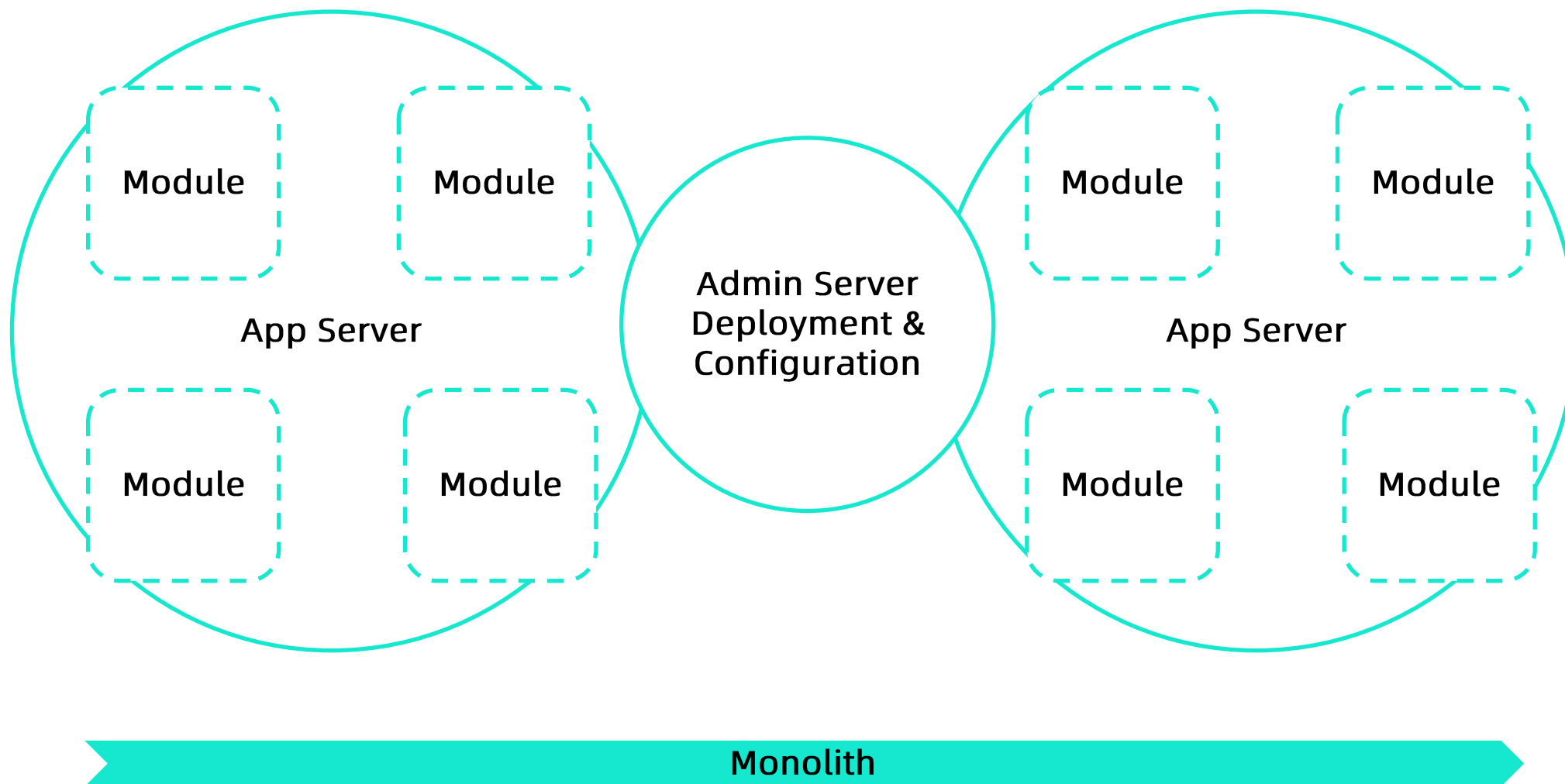


目录

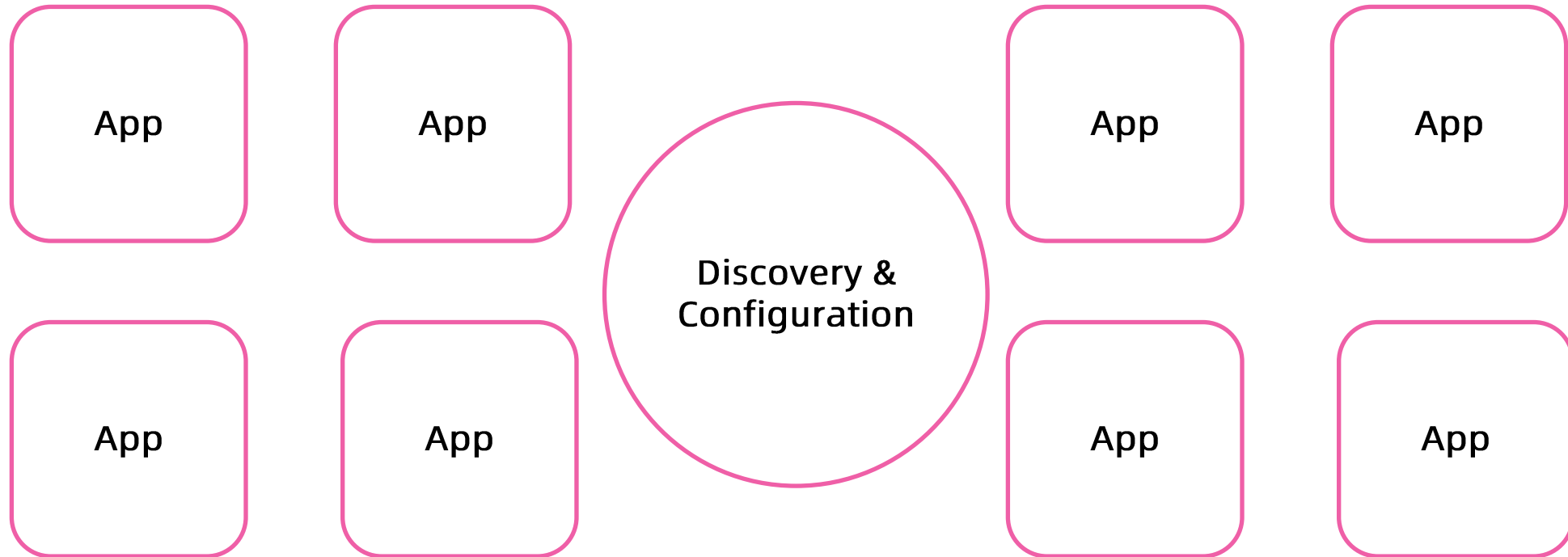
1. 微服务架构的演变
2. 微服务到服务网格还缺什么？
3. 深入理解数据平面 Envoy
4. Istio 流量管理
5. 跟踪采样

1. 微服务架构的演变

Evolution



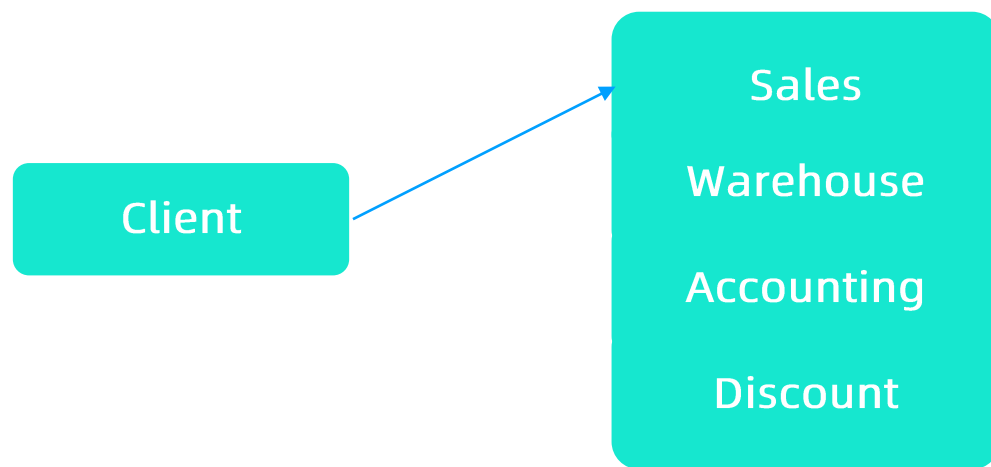
Evolution



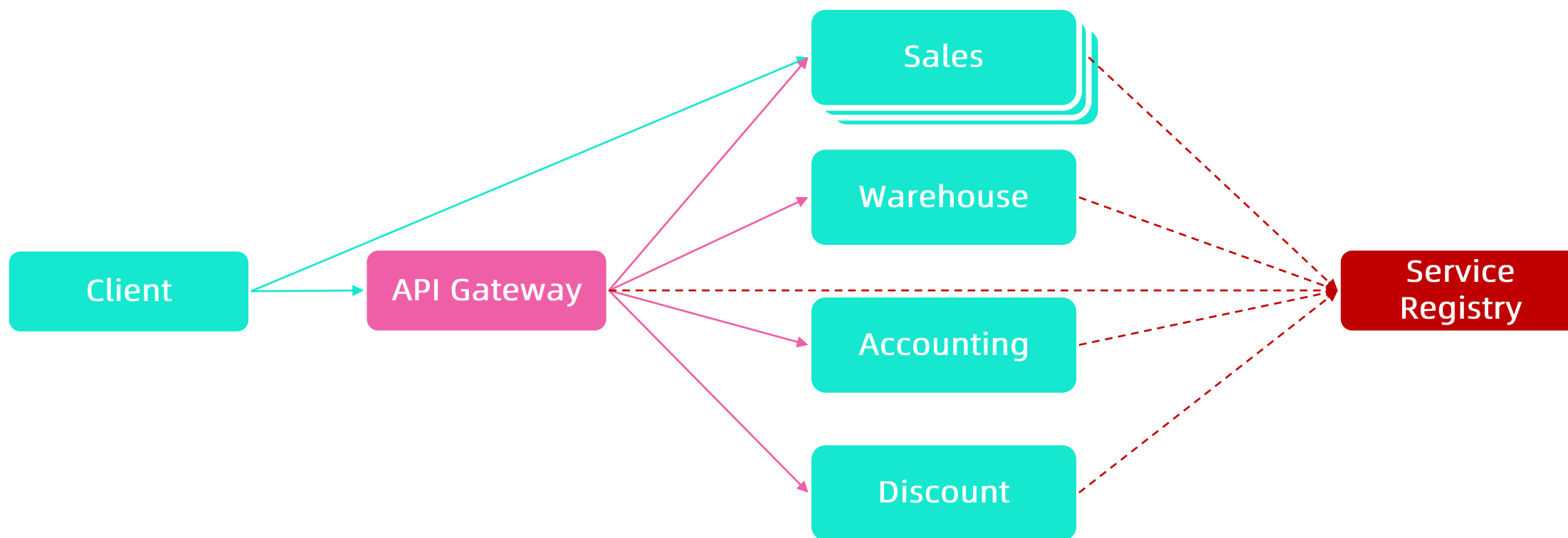
Microservice

从单块系统到微服务系统的演进

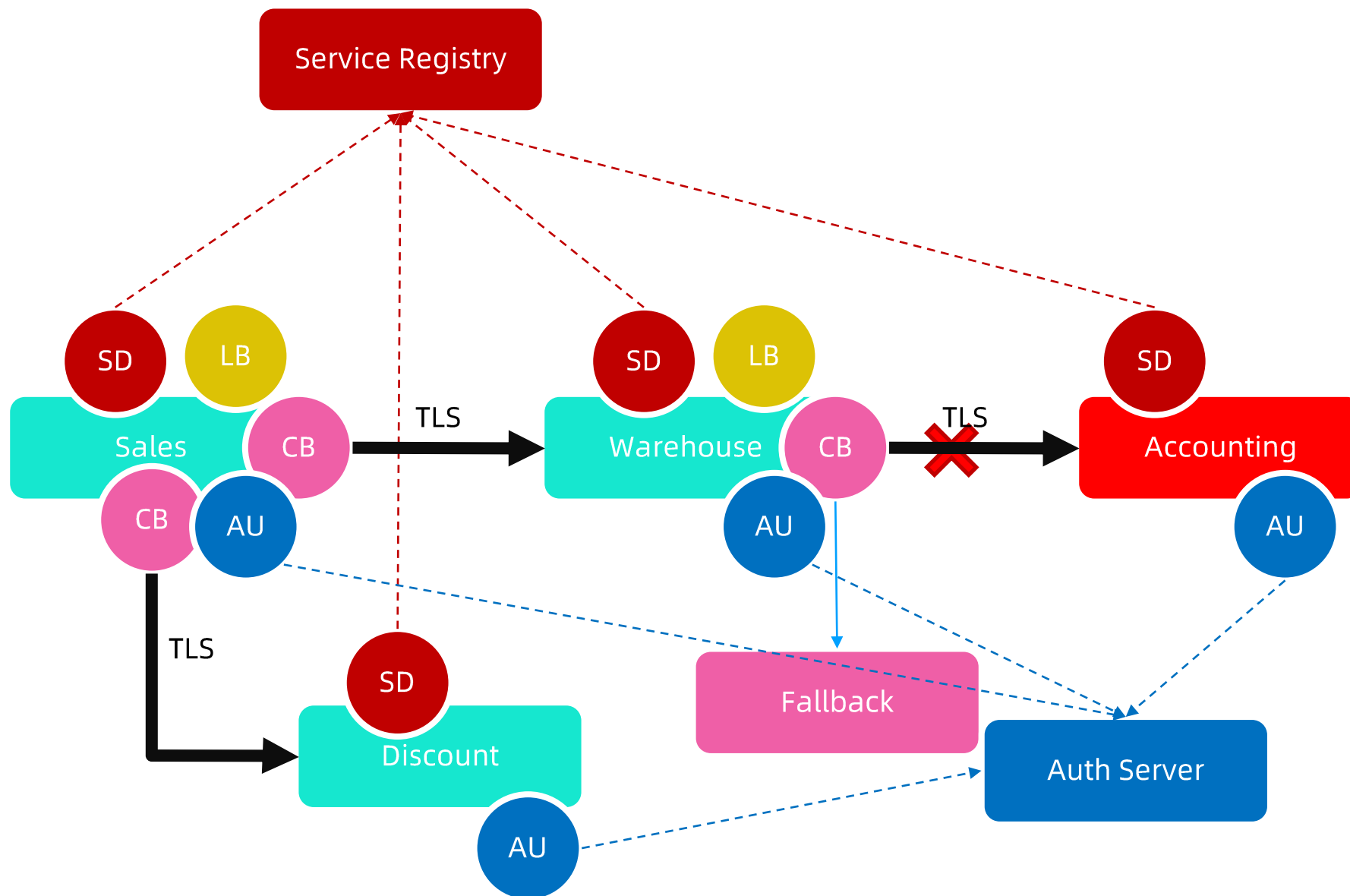
Monolith



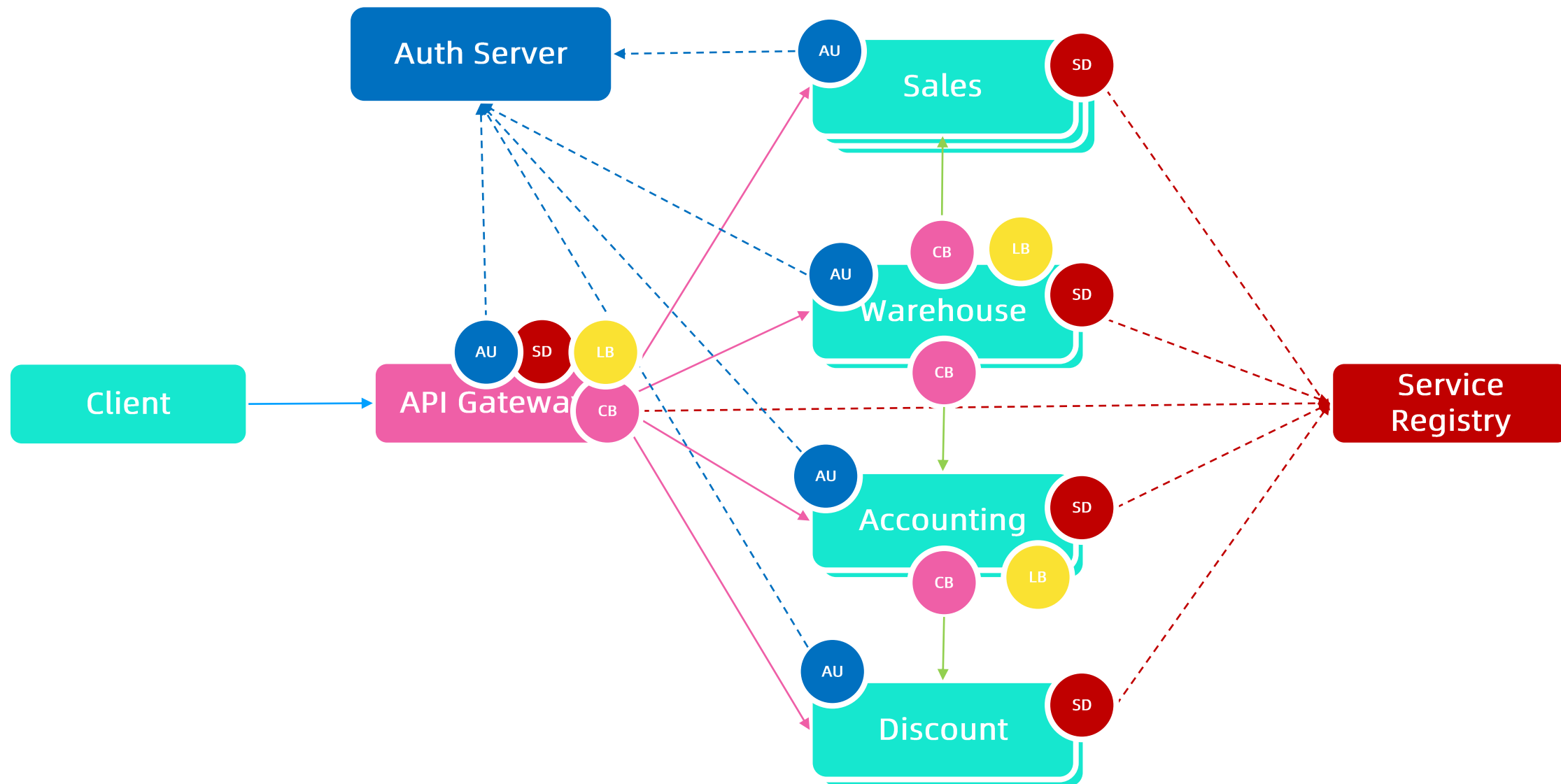
微服务架构的演进



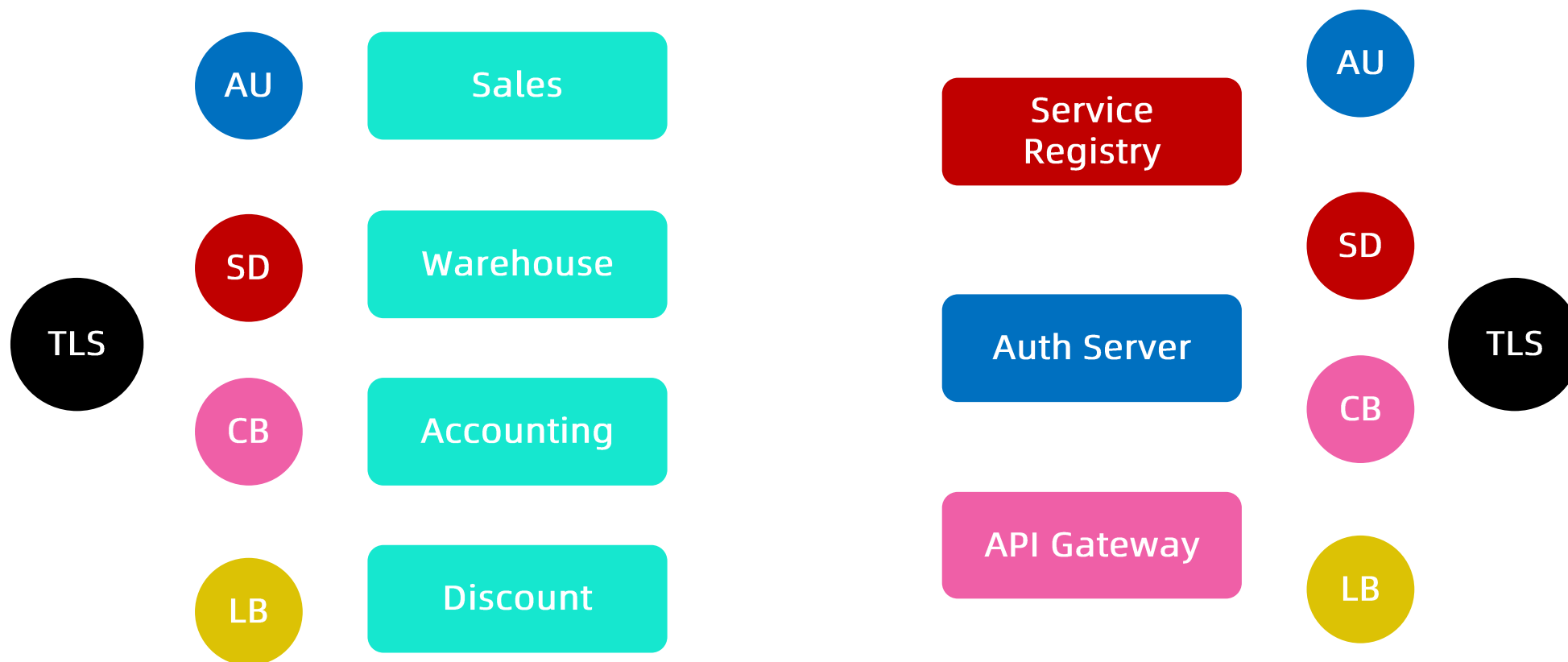
典型的微服务业务场景



更完整的微服务架构

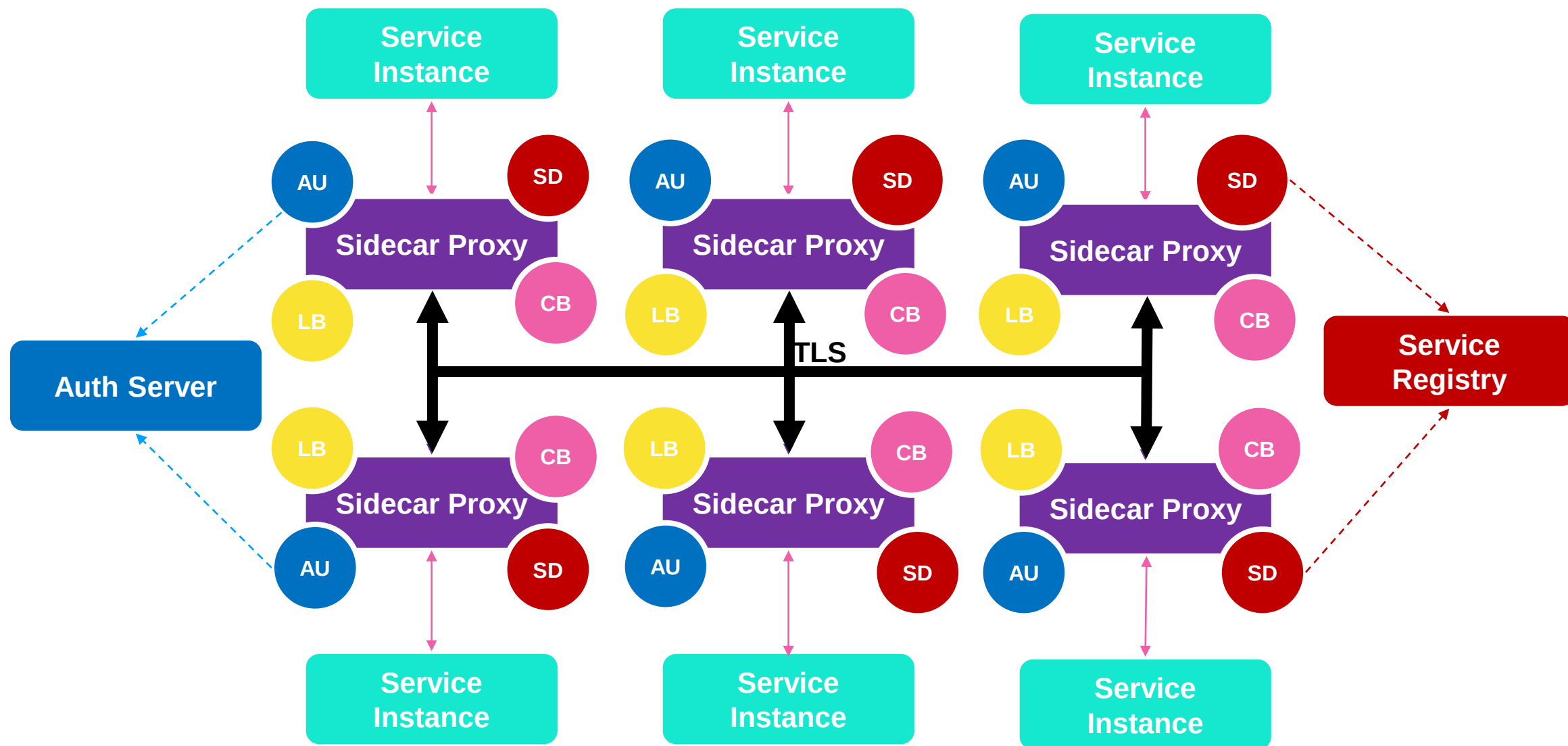


系统边界

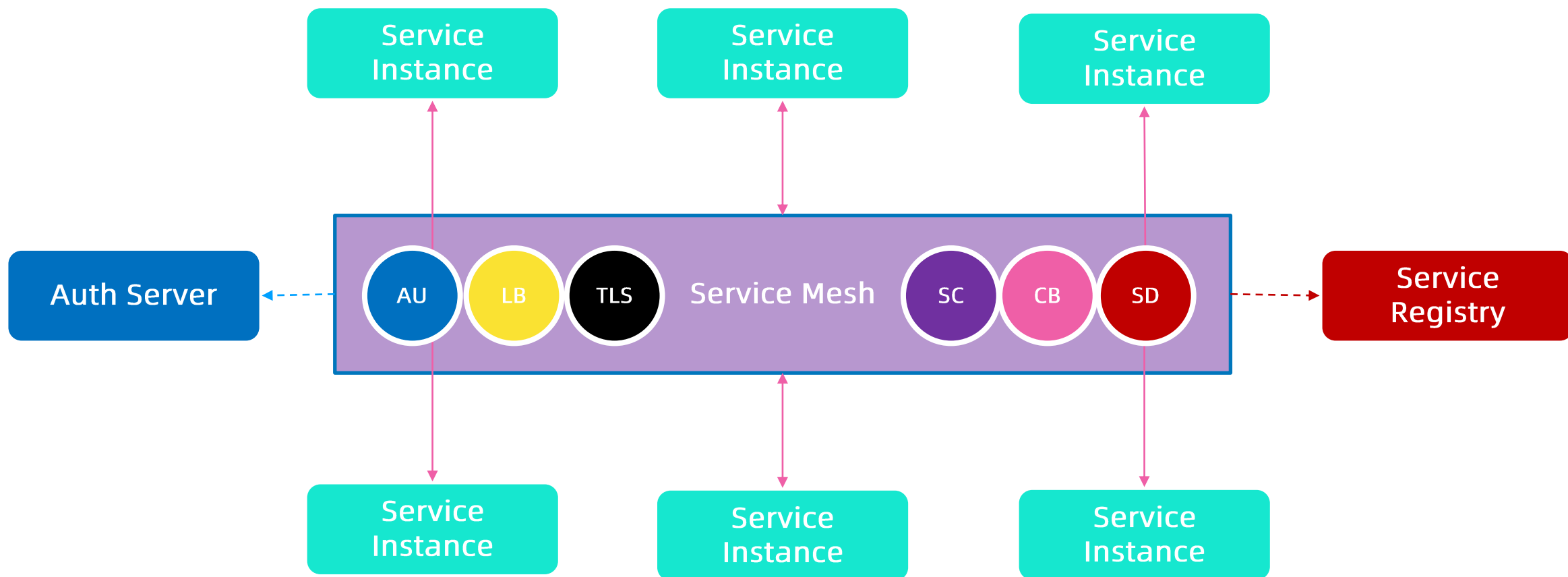


2. 微服务到服务网格还缺什么？

Sidecar 的工作原理



Service Mesh



Service Mesh

- 适应性
 - 熔断
 - 重试
 - 超时处理
 - 失败处理
 - 负载均衡
 - Failover
- 服务发现
 - 路由
- 安全和访问控制
 - TLS 和证书管理
- 可观察行
 - Metrics
 - 监控
 - 分布式日志
 - 分布式 tracing
- 部署
 - 容器
- 通讯
 - HTTP
 - WS
 - gRPC
 - TCP

微服务的优劣

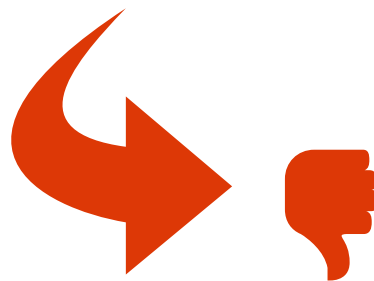
将基础架构逻辑从业务代码中剥离出来

- 分布式 tracing
- 日志

自由选择技术栈

帮助业务开发部门只关注业务逻辑

优势



劣势

复杂

- 更多的运行实例

可能带来额外的网络跳转

- 每个服务调用都要经过 Sidecar

解决了一部分问题，同时要付出代价

- 依然要处理复杂路由，类型映射，与外部系统整合等方面问题

不解决业务逻辑或服务整合，服务组合等问题

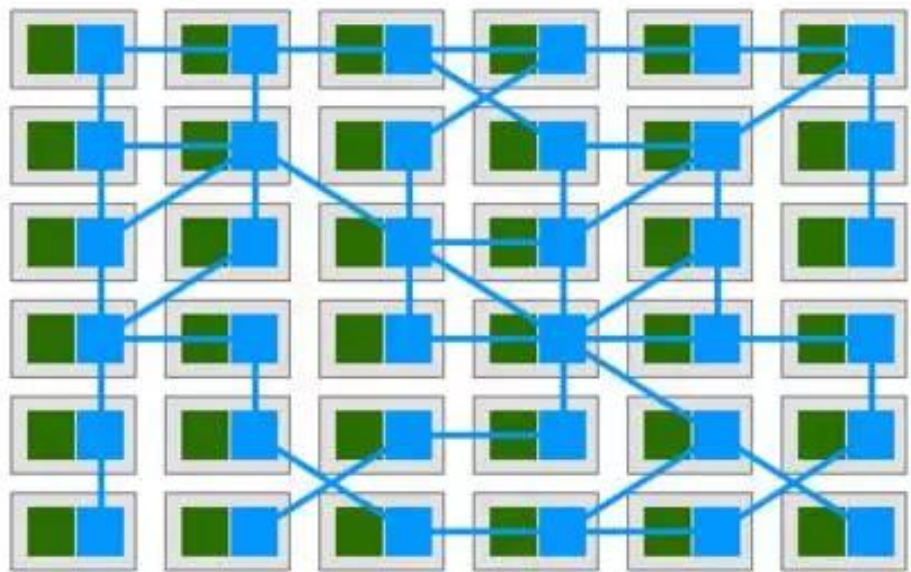
服务网格可选方案

	Istio	Linkerd	Linkerd2	Consul Connect
Model	Sidecar	Node Agent	Sidecar	Sidecar
Platform	Kubernetes	Any	Kubernetes	Any
language	Go	JVM	Go / Rust	Go
Protocol	HTTP1.1 / HTTP2 / gRPC / TCP	HTTP1.1 / HTTP2 / gRPC	HTTP1.1 / HTTP2 / gRPC / TCP	TCP
Default Data Plane	Envoy (supports others)	Native	Native	Native (or Envoy)
Sidecar Injection	Yes	No	No	No
Encryption	Yes	Yes	Experimental	Yes
Traffic Control	label/content based routing, traffic shifting	Dynamic request routing, traffic shifting, per request routing	?	static upstream, prepared query, http api / dns with native integration
Resilience	timeouts, retries, connection pools, outlier detection	timeouts, retries, deadlines, circuit breaking	?	Pluggable
Prometheus Integration	Yes	Yes	Yes	No
Tracing Integration	Jaeger	Zipkin	None	Pluggable
Host to Host auth	Service Accounts	TLS Mutual Auth	No	Consul ACL
Agent Caching	Yes	No	No	Yes
Secure connection outside cluster	No	Yes	No	Yes
Complexity	High	High	Low	Low
Paid Support	No	Yes	Yes	Yes

什么是服务网格

服务网格（Service Mesh）这个术语通常用于描述构成这些应用程序的微服务网络以及应用之间的交互。随着规模和复杂性的增长，服务网格越来越难以理解和管理。

它的需求包括服务发现、负载均衡、故障恢复、指标收集和监控以及通常更加复杂的运维需求，例如 A/B 测试、金丝雀发布、限流、访问控制和端到端认证等。



为什么要使用 Istio?

- HTTP、gRPC、WebSocket 和 TCP 流量的自动负载均衡。
- 通过丰富的路由规则、重试、故障转移和故障注入，可以对流量行为进行细粒度控制。
- 可插入的策略层和配置 API，支持访问控制、速率限制和配额。
- 对出入集群入口和出口中所有流量的自动度量指标、日志记录和跟踪。
- 通过强大的基于身份的验证和授权，在集群中实现安全的服务间通信。

Istio 功能概览



请求路由



服务发现和
负载均衡



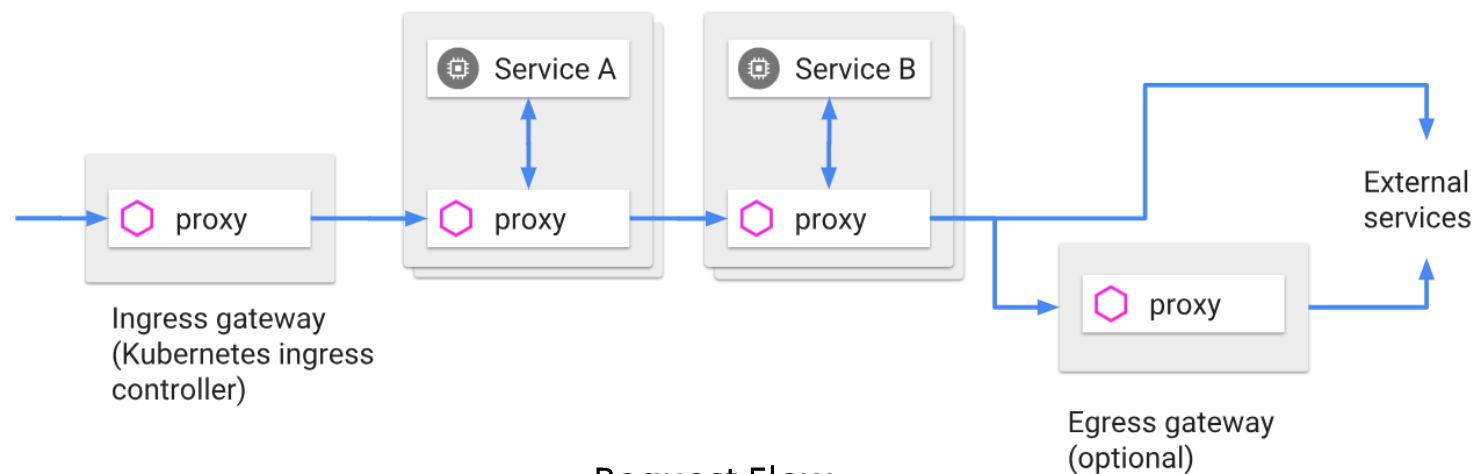
故障处理



故障注入



规则配置

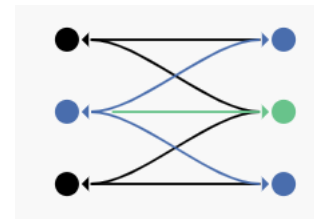


Request Flow

流量管理

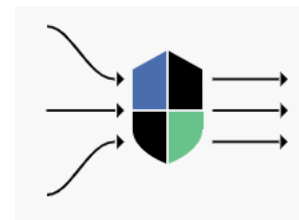
连接

- 通过简单的规则配置和流量路由，可以控制服务之间的流量和 API 调用。Istio 简化了断路器、超时和重试等服务级别属性的配置，并且可以轻松设置 A/B 测试、金丝雀部署和基于百分比的流量分割的分阶段部署等重要任务。



控制

- 通过更好地了解流量和开箱即用的故障恢复功能，可以在问题出现之前先发现问题，使调用更可靠，并且使您的网络更加强大——无论您面临什么条件。



安全

- 使开发人员可以专注于应用程序级别的安全性。Istio 提供底层安全通信信道，并大规模管理服务通信的认证、授权和加密。使用 Istio，服务通信在默认情况下是安全的，它允许跨多种协议和运行时一致地实施策略——所有这些都很少或根本不需要应用程序更改。
- 虽然 Istio 与平台无关，但将其与 Kubernetes（或基础架构）网络策略结合使用，其优势会更大，包括在网络和应用层保护 Pod 间或服务间通信的能力。

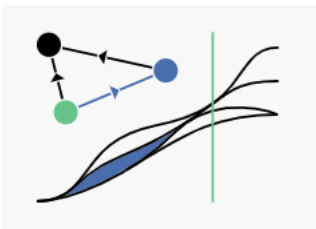


可观察性

Istio 生成以下类型的遥测数据，以提供对整个服务网格的可观察性：

- **指标：**Istio 基于 4 个监控的黄金标识（延迟、流量、错误、饱和）生成了一系列服务指标。Istio 还为网格控制平面提供了更详细的指标。除此以外还提供了一组默认的基于这些指标的网格监控仪表板。
- **分布式追踪：**Istio 为每个服务生成分布式追踪 span，运维人员可以理解网格内服务的依赖和调用流程。
- **访问日志：**当流量流入网格中的服务时，Istio 可以生成每个请求的完整记录，包括源和目标的元数据。此信息使运维人员能够将服务行为的审查控制到单个工作负载实例的级别。

所有这些功能可以更有效地设置、监控和实施服务上的 SLO，快速有效地检测和修复问题。



Istio 架构演进

数据平面

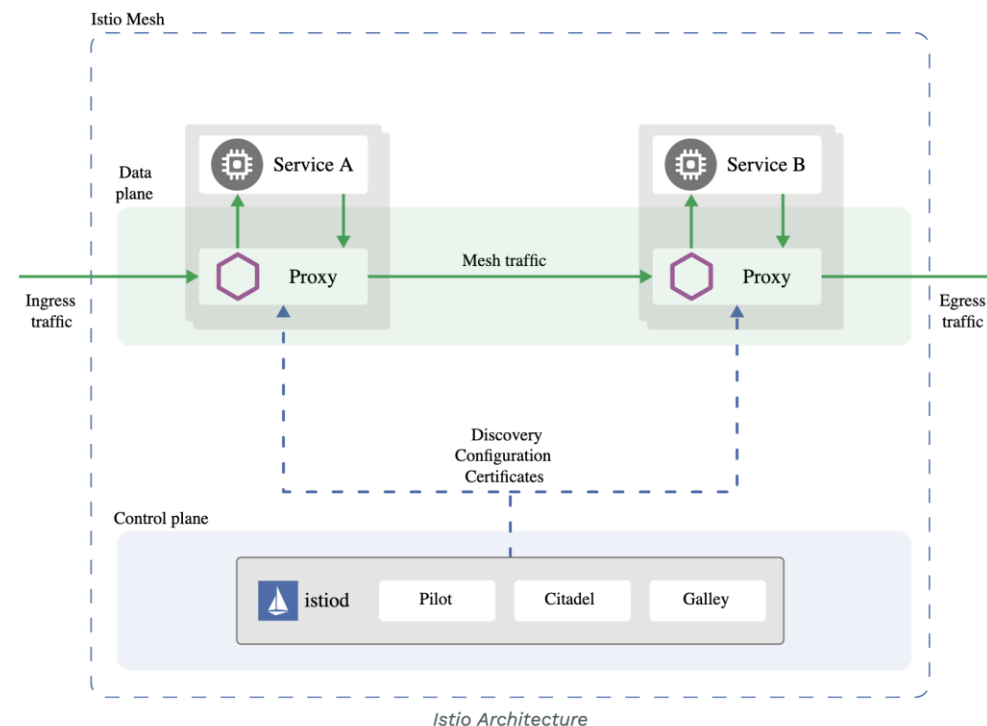
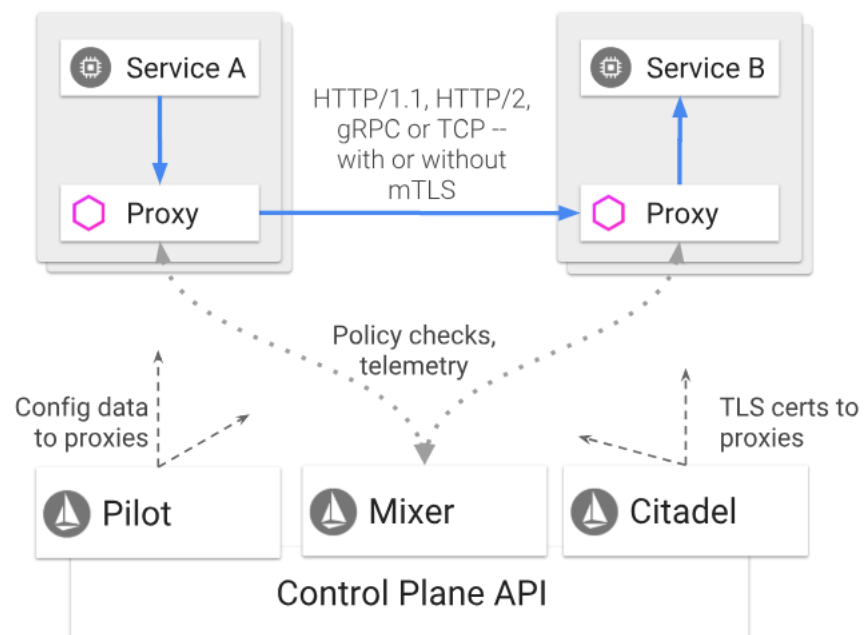
- 由一组以 Sidecar 方式部署的智能代理（Envoy）组成。这些代理可以调节和控制微服务及 Mixer 之间所有的网络通信。

控制平面

- 负责管理和配置代理来路由流量。此外控制平面配置 Mixer 以实施策略和收集遥测数据。

架构演进

- 从微服务回归单体



设计目标

最大化透明度

- Istio 将自身自动注入到服务间所有的网络路径中，运维和开发人员只需付出很少的代价就可以从中受益。
- Istio 使用 Sidecar 代理来捕获流量，并且在尽可能的地方自动编程网络层，以路由流量通过这些代理，而无需对已部署的应用程序代码进行任何改动。
- 在 Kubernetes 中，代理被注入到 Pod 中，通过编写 iptables 规则来捕获流量。注入 Sidecar 代理到 Pod 中并且修改路由规则后，Istio 就能够调解所有流量。
- 所有组件和 API 在设计时都必须考虑性能和规模。

增量

- 预计最大的需求是扩展策略系统，集成其他策略和控制来源，并将网格行为信号传播到其他系统进行分析。策略运行时支持标准扩展机制以便插入到其他服务中。

设计目标

可移植性

- 将基于 Istio 的服务移植到新环境应该是轻而易举的，而使用 Istio 将一个服务同时部署到多个环境中也是可行的（例如，在多个云上进行冗余部署）。

策略一致性

- 在服务间的 API 调用中，策略的应用使得可以对网格间行为进行全面的控制，但对于无需在 API 级别表达的资源来说，对资源应用策略也同样重要。
- 因此，策略系统作为独特的服务来维护，具有自己的 API，而不是将其放到代理 /sidecar 中，这容许服务根据需要直接与其集成。

3. 深入理解数据平面 Envoy

主流七层代理的比较

Envoy		Nginx		HA Proxy
HTTP/2	对 HTTP/2 有最完整的支持，同时支持 upstream 和 downstream HTTP/2.	从1.9.5开始有限支持 HTTP/2 只在 upstream server 支持 HTTP/2, downstream 依然是1.1.		不支持 HTTP/2.
Rate Limit	通过插件支持限流	支持基于配置的限流，只支持基于源IP的 限流		
ACL	基于插件实现四层 ACL	基于源 / 目标地址实现 ACL		
Connection draining	支持 hot reload，并且通过 share memory 实现 connection draining 的功能	Nginx Plus 收费版支持 connection draining		支持热启动，但不保证 丢弃连接

Envoy 的优势

性能：

- 在具备大量特性的同时，Envoy 提供极高的吞吐量和低尾部延迟差异，而 CPU 和 RAM 消耗却相对较少。

可扩展性：

- Envoy 在 L4 和 L7 都提供了丰富的可插拔过滤器能力，使用户可以轻松添加 开源版本中没有的功能。

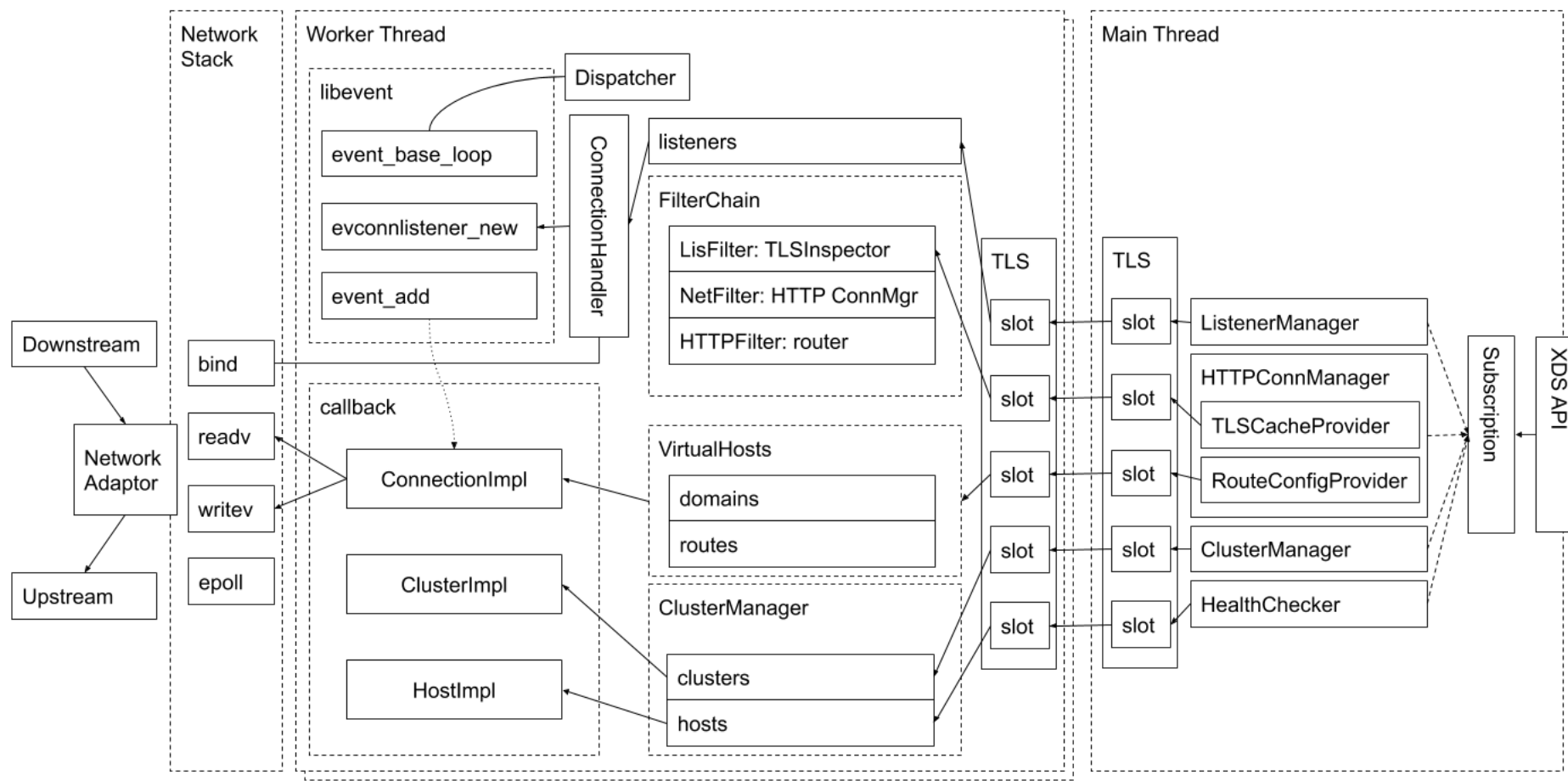
API可配置性：

- Envoy 提供了一组可以通过控制平面服务实现的管理 API。如果控制平面实现所有的 API，则可以使用通用引导配置在整个基础架构上运行 Envoy。所有进一步的配置更改通过管理服务器以无缝方式动态传送，因此 Envoy 从不需要重新启动。这使得 Envoy 成为通用数据平面，当它与一个足够复杂的控制平面相结合时，会极大的降低整体运维的复杂性。

Envoy 线程模式

- Envoy 采用单进程多线程模式：
 - 主线程负责协调；
 - 子线程负责监听过滤和转发。
- 当某连接被监听器接受，那么该连接的全部生命周期会与某线程绑定。
- Envoy 基于非阻塞模式（Epoll）。
- 建议 Envoy 配置的 worker 数量与 Envoy 所在的硬件线程数一致。

Envoy 架构



v1 API 的缺点和 v2 的引入

v1 API 仅使用 JSON/REST，本质上是轮询。这有几个缺点：

- 尽管 Envoy 在内部使用的是 JSON 模式，但 API 本身并不是强类型，而且安全实现它们的通用服务器也很难。
- 虽然轮询工作在实践中是很正常的用法，但更强大的控制平面更喜欢 streaming API，当其就绪后，可以将更新推送给每个 Envoy。这可以将更新传播时间从 30-60 秒降低到 250-500 毫秒，即使在极其庞大的部署中也是如此。

v2 API 具有以下属性：

- 新的 API 模式使用 proto3 指定，并同时以 gRPC 和 REST + JSON/YAML 端点实现。
- 它们被定义在一个名为 envoy-api 的新的专用源代码仓库中。proto3 的使用意味着这些 API 是强类型的，同时仍然通过 proto3 的 JSON/YAML 表示来支持 JSON/YAML 变体。
- 专用存储仓库的使用意味着项目可以更容易的使用 API 并用 gRPC 支持的所有语言生成存根（实际上，对于希望使用它的用户，我们将继续支持基于 REST 的 JSON/YAML 变体）。

xDS – Envoy 的发现机制

Endpoint Discovery Service (EDS):

- 这是 v1 SDS API 的替代品。此外，gRPC 的双向流性质将允许将负载/健康信息报告回管理服务，为将来的全局负载均衡功能开启大门。

Cluster Discovery Service (CDS):

- 和 v1 没有实质性变化。

Route Discovery Service (RDS):

- 和 v1 没有实质性变化。

Listener Discovery Service (LDS):

- 和 v1 的唯一主要变化是：现在允许监听器定义多个并发过滤栈，这些过滤栈可以基于一组监听器路由规则（例如，SNI，源/目的地 IP 匹配等）来选择。这是处理“原始目的地”策略路由的更简洁的方式，这种路由是透明数据平面解决方案（如 Istio）所需要的。

xDS – Envoy 的发现机制

Secret Discovery Service (SDS):

- 一个专用的 API 来传递 TLS 密钥材料。这将解耦通过 LDS/CDS 发送主要监听器、集群配置和通过专用密钥管理系统发送密钥素材。

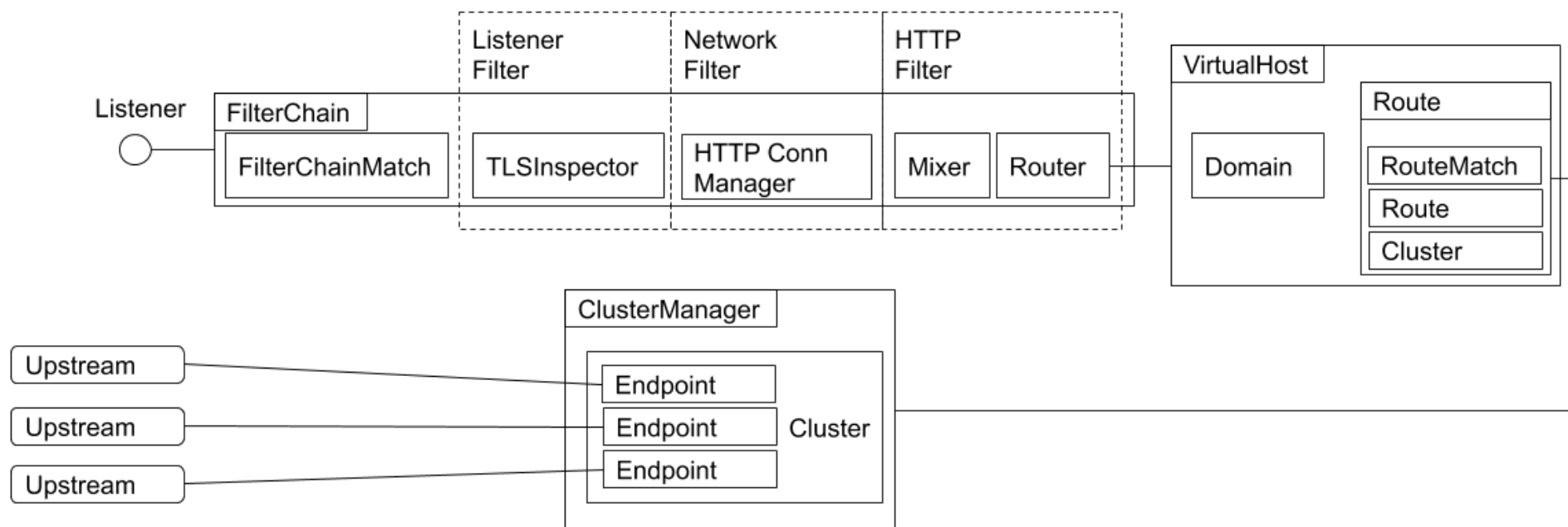
Health Discovery Service (HDS):

- 该 API 将允许 Envoy 成为分布式健康检查网络的成员。中央健康检查服务可以使用一组 Envoy 作为健康检查终点并将状态报告回来，从而缓解 N^2 健康检查问题，这个问题指的是其间的每个 Envoy 都可能需要对每个其他 Envoy 进行健康检查。

Aggregated Discovery Service (ADS):

- 总的来说，Envoy 的设计是最终一致的。这意味着默认情况下，每个管理 API 都并发运行，并且不会相互交互。在某些情况下，一次一个管理服务器处理单个 Envoy 的所有更新是有益的（例如，如果需要对更新进行排序以避免流量下降）。此 API 允许通过单个管理服务器的单个 gRPC 双向流对所有其他 API 进行编组，从而实现确定性排序。

Envoy 的过滤器模式



4. Isito 流量管理

流量管理

- Gateway
- VirtualService
- DestinationRule
- ServiceEntry
- WorkloadEntry
- Sidecar

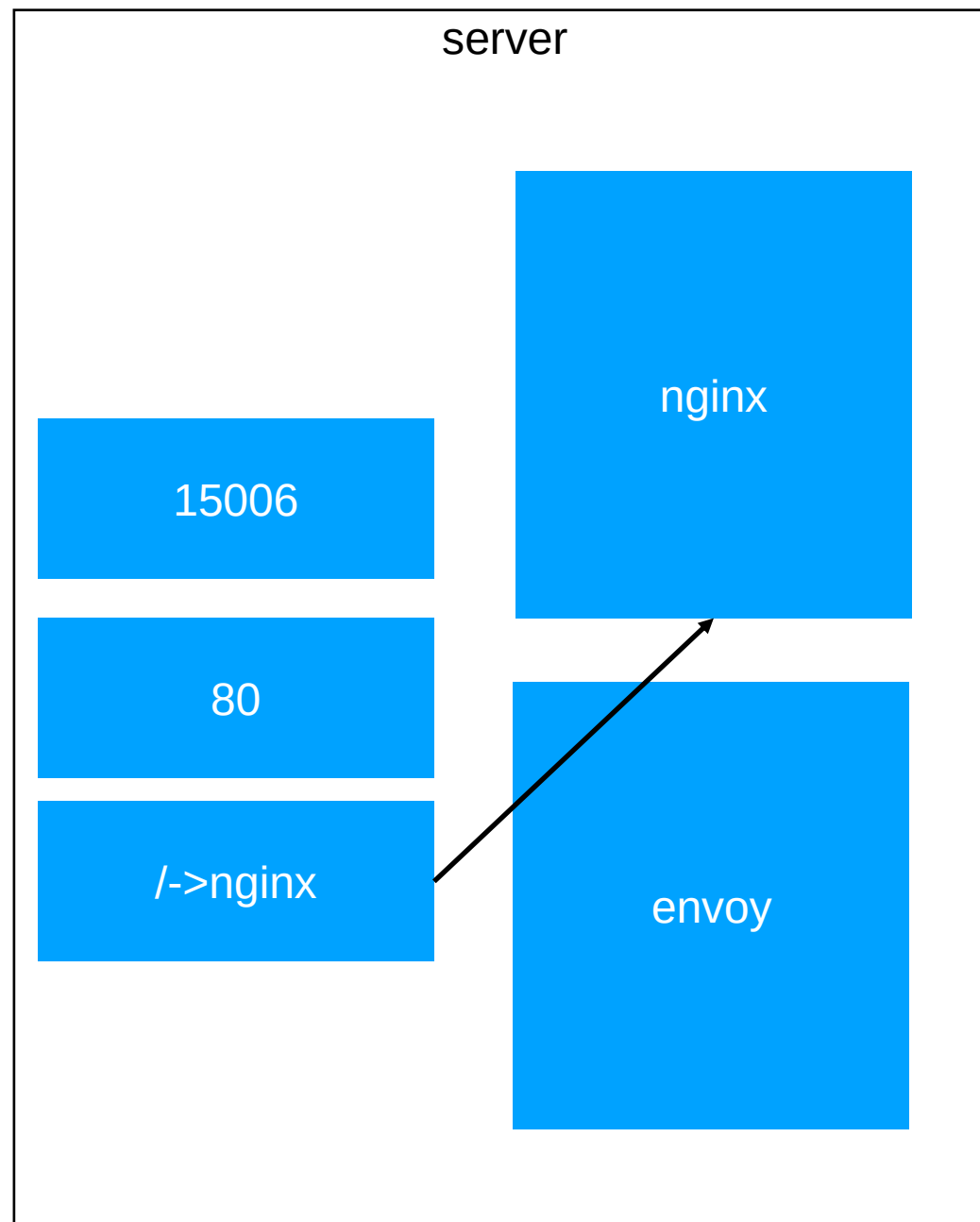
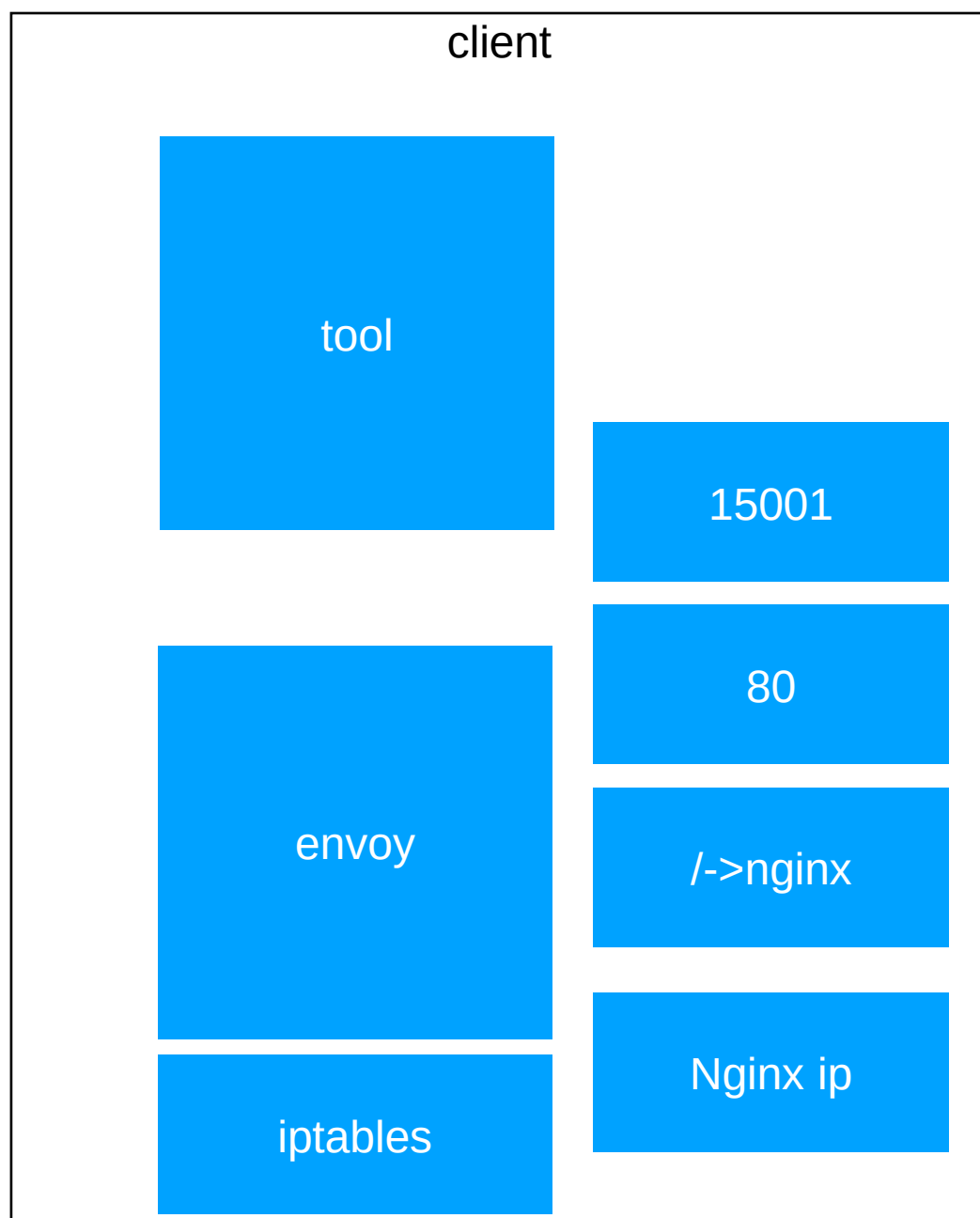
Istio 的流量劫持机制

为用户应用注入 Sidecar

- 自动注入
- 手动注入
 - `istioctl kube-inject -f yaml/istio-bookinfo/bookinfo.yaml`

注入后的结果

- 注入了 init-container istio-init
 - `istio-iptables -p 15001 -z 15006-u 1337 -m REDIRECT -i * -x -b 9080 -d 15090,15021,15020`
- 注入了 sidecar container istio-proxy



Init container

将应用容器的所有流量都转发到 Envoy 的 15001 端口。

使用 istio-proxy 用户身份运行，UID 为 1337，即 Envoy 所处的用户空间，这也是 istio-proxy 容器默认使用的用户，见 YAML 配置中的 runAsUser 字段。

使用默认的 REDIRECT 模式来重定向流量。

将所有出站流量都重定向到 Envoy 代理。

将所有访问 9080 端口（即应用容器 productpage 的端口）的流量重定向到 Envoy 代理。

```
//所有入站TCP流量走ISTIO_INBOUND
-A PREROUTING -p tcp -j ISTIO_INBOUND

// 所有出站TCP流量走ISTIO_OUTPUT
-A OUTPUT -p tcp -j ISTIO_OUTPUT

// 忽略ssh, health check等端口
-A ISTIO_INBOUND -p tcp -m tcp --dport 22 -j RETURN
-A ISTIO_INBOUND -p tcp -m tcp --dport 15090 -j RETURN
-A ISTIO_INBOUND -p tcp -m tcp --dport 15021 -j RETURN
-A ISTIO_INBOUND -p tcp -m tcp --dport 15020 -j RETURN
// 所有入站TCP流量走ISTIO_IN_REDIRECT
-A ISTIO_INBOUND -p tcp -j ISTIO_IN_REDIRECT

// TCP流量转发至15006端口
-A ISTIO_IN_REDIRECT -p tcp -j REDIRECT --to-ports 15006

// loopback passthrough
-A ISTIO_OUTPUT -s 127.0.0.6/32 -o lo -j RETURN
// 从loopback口出来, 目标非本机地址, owner是envoy, 交由ISTIO_IN_REDIRECT处理
-A ISTIO_OUTPUT ! -d 127.0.0.1/32 -o lo -m owner --uid-owner 1337 -j
ISTIO_IN_REDIRECT
// 从lo口出来, owner非envoy, return
-A ISTIO_OUTPUT -o lo -m owner ! --uid-owner 1337 -j RETURN
// owner是envoy, return
-A ISTIO_OUTPUT -m owner --uid-owner 1337 -j RETURN
-A ISTIO_OUTPUT ! -d 127.0.0.1/32 -o lo -m owner --gid-owner 1337 -j
ISTIO_IN_REDIRECT
-A ISTIO_OUTPUT -o lo -m owner ! --gid-owner 1337 -j RETURN
-A ISTIO_OUTPUT -m owner --gid-owner 1337 -j RETURN
-A ISTIO_OUTPUT -d 127.0.0.1/32 -j RETURN
// 如以上规则都不匹配, 则交给ISTIO_REDIRECT处理
-A ISTIO_OUTPUT -j ISTIO_REDIRECT

-A ISTIO_REDIRECT -p tcp -j REDIRECT --to-ports 15001
```

Sidecar container

Istio 会生成以下监听器：

- 0.0.0.0:15001 上的监听器接收进出 Pod 的所有流量，然后将请求移交给虚拟监听器。
- 每个 service IP 一个虚拟监听器，每个出站 TCP/HTTPS 流量一个非 HTTP 监听器。
- 每个 Pod 进站流量暴露的端口一个虚拟监听器。
- 每个出站 HTTP 流量的 HTTP 0.0.0.0 端口一个虚拟监听器。

```
istioctl proxy-config listeners productpage-v1-8b96c8794-bttd -n bookinfo --port 15001 -ojson
```

```
{
  "name": "virtual",
  "address": {
    "socketAddress": {
      "address": "0.0.0.0",
      "portValue": 15001
    }
  },
  "filterChains": [
    {
      "filters": [
        {
          "name": "envoy.tcp_proxy",
          "config": {
            "cluster": "BlackHoleCluster",
            "stat_prefix":
"BlackHoleCluster"
          }
        }
      ]
    }
  ],
  "useOriginalDst": true
}
```


Sidecar container

我们的请求是到 9080 端口的 HTTP 出站请求，这意味着它被切换到 0.0.0.0:9080 虚拟监听器。然后，此监听器在其配置的 RDS 中查找路由配置。在这种情况下，它将查找由 Pilot 配置的 RDS 中的路由 9080（通过 ADS）。

```
istioctl proxy-config listeners productpage-v1-8b96c8794-btddd -n bookinfo --port 9080  
--address 0.0.0.0 -ojson
```

```
netstat -na|grep LISTEN  
tcp        0      0 127.0.0.1:15000      0.0.0.0:*             LISTEN  
tcp        0      0 0.0.0.0:15001        0.0.0.0:*             LISTEN  
tcp6       0      0 :::9080              :::*                  LISTEN
```

```
"rds": {  
  "config_source": {  
    "ads": {}  
  },  
  "route_config_name": "9080"  
}
```

Sidecar container

9080 路由配置仅为每个服务提供虚拟主机。我们的请求正在前往 reviews 服务，因此 Envoy 将选择我们的请求与域匹配的虚拟主机。一旦在域上匹配，Envoy 会查找与请求匹配的第一条路径。在这种情况下，我们没有任何高级路由，因此只有一条路由匹配所有内容。

这条路由告诉 Envoy 将请求发送到
outbound|9080||reviews.default.svc.cluster.local
集群。

```
istioctl proxy-config routes productpage-v1-  
8b96c8794-bttd --name 9080 -o json -n  
bookinfo
```

```
"name": "9080",  
"virtualHosts": [  
  {  
    "name": "reviews.bookinfo.svc.cluster.local:9080",  
    "domains": [  
      "reviews.bookinfo.svc.cluster.local",  
      "reviews.bookinfo.svc.cluster.local:9080",  
      "reviews",  
      "reviews:9080",  
      "reviews.bookinfo.svc.cluster",  
      "reviews.bookinfo.svc.cluster:9080",  
      "reviews.bookinfo.svc",  
      "reviews.bookinfo.svc:9080",  
      "reviews.bookinfo",  
      "reviews.bookinfo:9080",  
      "192.168.143.232",  
      "192.168.143.232:9080"  
    ],  
    "routes": [  
      {  
        "match": {  
          "prefix": "/"  
        },  
        "route": {  
          "cluster":  
            "outbound|9080||reviews.bookinfo.svc.cluster.local",  
          "timeout": "0.000s",  
          "maxGrpcTimeout": "0.000s"  
        }  
      }  
    ]  
  }  
]
```

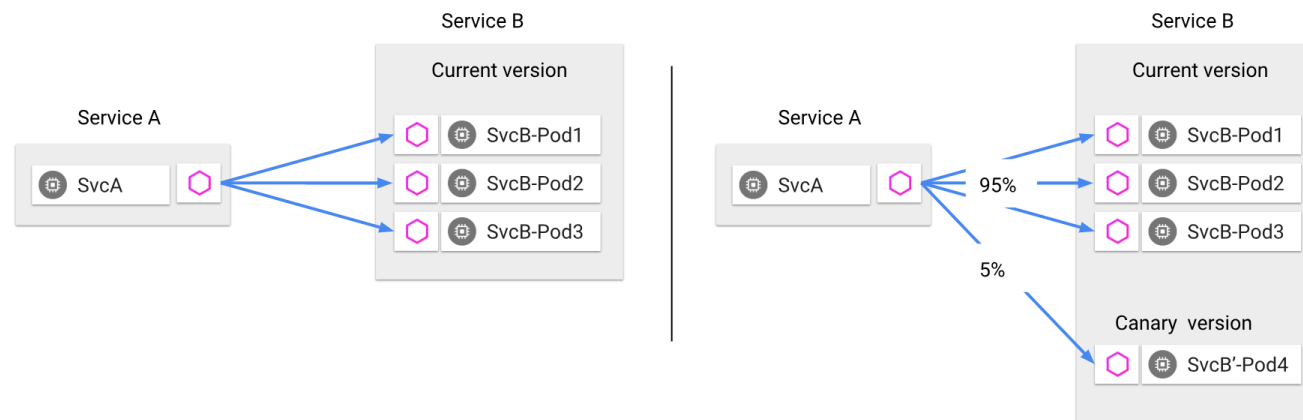
Sidecar container

此集群配置为从 Pilot（通过 ADS）检索关联的端点。因此，Envoy 将使用 serviceName 字段作为密钥来查找端点列表并将请求代理到其中一个端点。

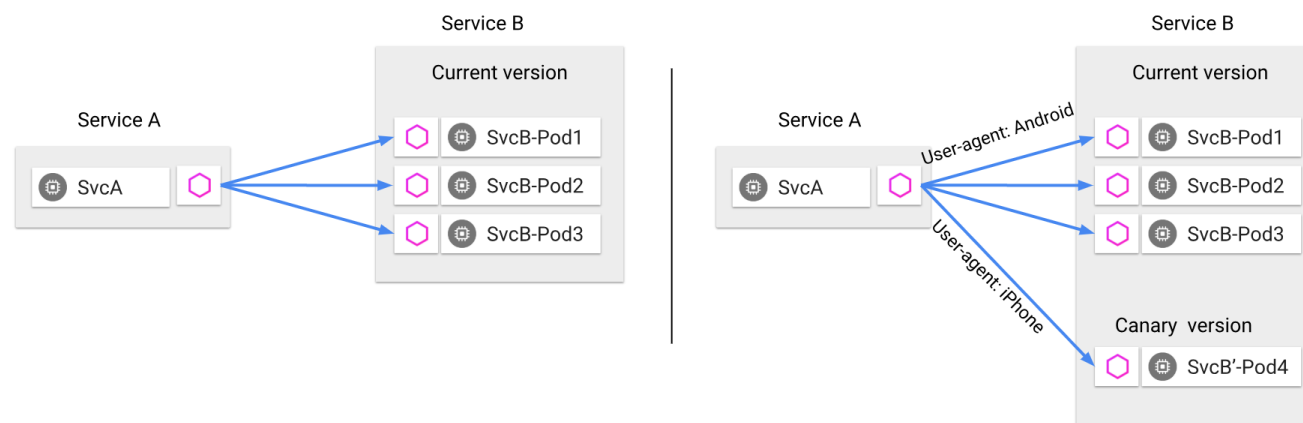
```
istioctl proxy-config clusters productpage-v1-8b96c8794-bttd --fqdn  
reviews.bookinfo.svc.cluster.local -n bookinfo -ojson
```

```
"name": "outbound|9080|reviews.bookinfo.svc.cluster.local",  
"type": "EDS",  
"edsClusterConfig": {  
  "edsConfig": {  
    "ads": {}  
  },  
  "serviceName": "outbound|9080|reviews.bookinfo.svc.cluster.local"  
},  
"connectTimeout": "1.000s",  
"circuitBreakers": {  
  "thresholds": [  
    {}  
  ]  
}
```

流量管理



Traffic splitting decoupled from infrastructure scaling - proportion of traffic routed to a version is independent of number of instances supporting the version



Content-based traffic steering - The content of a request can be used to determine the destination of a request

请求路由

特定网格中服务的规范表示由 Pilot 维护。服务的 Istio 模型和在底层平台（Kubernetes、Mesos 以及 Cloud Foundry 等）中的表达无关。特定平台的适配器负责从各自平台中获取元数据的各种字段，然后对服务模型进行填充。

Istio 引入了服务版本的概念，可以通过版本（v1、v2）或环境（staging、prod）对服务进行进一步的细分。这些版本不一定是不同的 API 版本：它们可能是部署在不同环境（prod、staging 或者 dev 等）中的同一服务的不同迭代。使用这种方式的常见场景包括 A/B 测试或金丝雀部署。

Istio 的流量路由规则可以根据服务版本来对服务之间流量进行附加控制。

服务之间的通讯

服务的客户端不知道服务不同版本间的差异。它们可以使用服务的主机名或者 IP 地址继续访问服务。Envoy sidecar/ 代理拦截并转发客户端和服务器之间的所有请求和响应。

Istio 还为同一服务版本的多个实例提供流量负载均衡。可以在服务发现和负载均衡中找到更多信息。

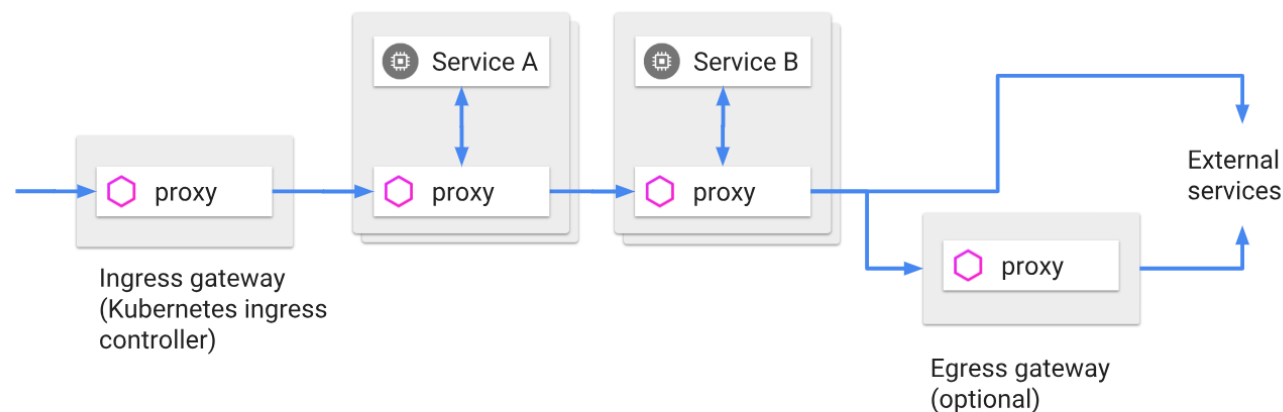
Istio 不提供 DNS。应用程序可以尝试使用底层平台（kube-dns、mesos-dns 等）中存在的 DNS 服务来解析 FQDN。

Ingress 和 Egress

Istio 假定进入和离开服务网络的所有流量都会通过 Envoy 代理进行传输。

通过将 Envoy 代理部署在服务之前，运维人员可以针对面向用户的服务进行 A/B 测试、部署金丝雀服务等。

类似地，通过使用 Envoy 将流量路由到外部 Web 服务（例如，访问 Maps API 或视频服务 API）的方式，运维人员可以为这些服务添加超时控制、重试、断路器等功能，同时还能从服务连接中获取各种细节指标。



服务发现和负载均衡

Istio 负载均衡服务网格中实例之间的通信。

Istio 假定存在服务注册表，以跟踪应用程序中服务的 **pod/VM**。它还假设服务的新实例自动注册到服务注册表，并且不健康的实例将被自动删除。诸如 Kubernetes、Mesos 等平台已经为基于容器的应用程序提供了这样的功能。为基于虚拟机的应用程序提供的解决方案就更多了。

Pilot 使用来自服务注册的信息，并提供与平台无关的服务发现接口。网格中的 Envoy 实例执行服务发现，并相应地动态更新其负载均衡池。

网格中的服务使用其 DNS 名称访问彼此。服务的所有 HTTP 流量都会通过 Envoy 自动重新路由。Envoy 在负载均衡池中的实例之间分发流量。虽然 Envoy 支持多种复杂的负载均衡算法，但 Istio 目前仅允许三种负载均衡模式：轮循、随机和带权重的最少请求。

服务发现和负载均衡

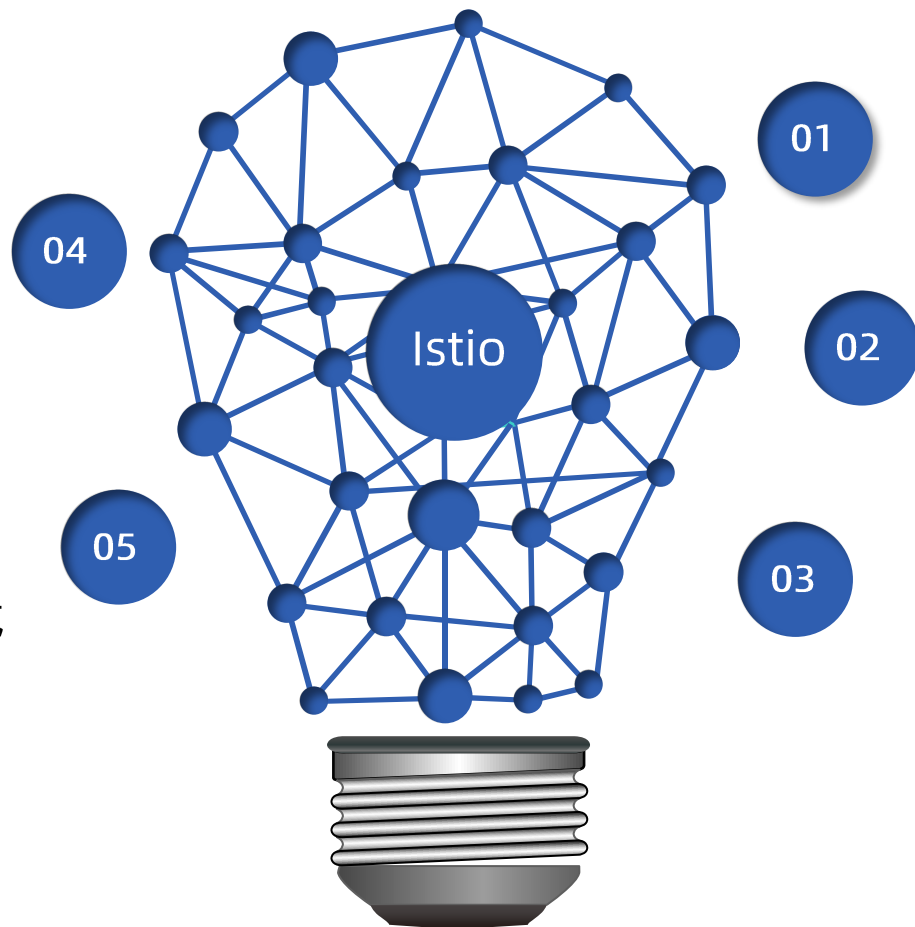
除了负载均衡外，Envoy 还会定期检查池中每个实例的运行状况。Envoy 遵循熔断器风格模式，根据健康检查 API 调用的失败率将实例分类为不健康和健康两种。当给定实例的健康检查失败次数超过预定阈值时，将会被从负载均衡池中弹出。类似地，当通过的健康检查数超过预定阈值时，该实例将被添加回负载均衡池。您可以在[处理故障](#)中了解更多有关 Envoy 的故障处理功能。

服务可以通过使用 HTTP 503 响应健康检查来主动减轻负担。在这种情况下，服务实例将立即从调用者的负载均衡池中删除。

故障处理

对负载均衡器成员的健康检查

细粒度的熔断机制，可以针对 Load Balancing Pool 中的每个成员设置规则



01 超时处理

02 基于超时预算的重试机制

03 基于并发连接和请求的流量控制

这些功能都可以 Istio 对象动态配置

微调

Istio 的流量管理规则允许运维人员为每个服务/版本设置故障恢复的全局默认值。然而，服务的消费者也可以通过特殊的 HTTP 头提供的请求级别值覆盖超时和重试的默认值。在 Envoy 代理的实现中，对应的 Header 分别是 `x-envoy-upstream-rq-timeout-ms` 和 `x-envoy-max-retries`。

故障注入

为什么需要错误注入：

- 微服务架构下，需要测试端到端的故障恢复能力。

Istio 允许在网络层面按协议注入错误来模拟错误，无需通过应用层面删除 Pod，或者人为在 TCP 层造成网络故障来模拟。

注入的错误可以基于特定的条件，可以设置出现错误的比例：

- Delay – 提高网络延时；
- Aborts – 直接返回特定的错误码。

规则配置

VirtualService 在 Istio 服务网格中定义路由规则，控制路由如何路由到服务上。

DestinationRule 是 VirtualService 路由生效后，配置应用与请求的策略集。

ServiceEntry 是通常用于在 Istio 服务网格之外启用对服务的请求。

Gateway 为 HTTP/TCP 流量配置负载均衡器，最常见的是在网格的边缘的操作，以启用应用程序的入口流量。

VirtualService

是在 Istio 服务网格内对服务的请求如何进行路由控制。

规则的目标描述

路由规则对应着一或多个用 `VirtualService` 配置指定的请求目的主机。这些主机可以是也可以不是实际的目标负载，甚至可以不是同一网格内可路由的服务。例如要给到 `reviews` 服务的请求定义路由规则，可以使用内部的名称 `reviews`，也可以用域名 `bookinfo.com`，`VirtualService` 可以定义这样的 `host` 字段：

`host` 字段用显示或者隐式的方式定义了一或多个完全限定名（FQDN）。上面的 `reviews`，会隐式的扩展成为特定的 FQDN，例如在 Kubernetes 环境中，全名会从 `VirtualService` 所在的集群和命名空间中继承而来（比如说 `reviews.default.svc.cluster.local`）。

`hosts:`

- `reviews`
- `bookinfo.com`

在服务之间分拆流量

例如下面的规则会把 25% 的 reviews 服务流量分配给 v2 标签；其余的 75% 流量分配给 v1。

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
      weight: 75
    - destination:
        host: reviews
        subset: v2
      weight: 25
```

超时和重试

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - route:
    - destination:
        host: ratings
        subset: v1
    timeout: 10s
```

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - route:
    - destination:
        host: ratings
        subset: v1
    retries:
      attempts: 3
      perTryTimeout: 2s
```

错误注入

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - fault:
      delay:
        percent: 10
        fixedDelay: 5s
    route:
    - destination:
        host: ratings
        subset: v1
```

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - fault:
      abort:
        percent: 10
        httpStatus: 400
    route:
    - destination:
        host: ratings
        subset: v1
```

条件规则

```
apiVersion:
networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: productpage
spec:
  hosts:
  - productpage
  http:
  - match:
    - uri:
        prefix: /api/v1
    ...
```

```
apiVersion:
networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews
  http:
  - match:
    - headers:
        end-user:
          exact: jason
    ...
```

```
apiVersion:
networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - match:
    sourceLabels:
      app: reviews
    ...
```

流量镜像

mirror 规则可以使 Envoy 截取所有 request, 并在转发请求的同时, 将 request 转发至 Mirror 版本, 同时在 Header 的 Host/Authority 加上 -shadow。

这些 mirror 请求会工作在 fire and forget 模式, 所有的 response 都会被废弃。

```
apiVersion:
networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: httpbin
spec:
  hosts:
  - httpbin
  http:
  - route:
    - destination:
        host: httpbin
        subset: v1
        weight: 100
    mirror:
      host: httpbin
      subset: v2
```

规则委托

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: bookinfo
spec:
  hosts:
    - "bookinfo.com"
  gateways:
    - mygateway
  http:
    - match:
        - uri:
            prefix: "/productpage"
      delegate:
        name: productpage
        namespace: nsA
    - match:
        - uri:
            prefix: "/reviews"
      delegate:
        name: reviews
        namespace: nsB
```

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: productpage
  namespace: nsA
spec:
  http:
    - match:
        - uri:
            prefix: "/productpage/v1/"
      route:
        - destination:
            host: productpage-v1.nsA.svc.cluster.local
    - route:
        - destination:
            host: productpage.nsA.svc.cluster.local
```

优先级

当对同一目标有多个规则时，会按照在 VirtualService 中的顺序进行应用，换句话说，列表中的第一条规则具有最高优先级。

当对某个服务的路由是完全基于权重的时候，就可以在单一规则中完成。另一方面，如果有多重条件（例如来自特定用户的请求）用来进行路由，就会需要不止一条规则。这样就出现了优先级问题，需要通过优先级来保证根据正确的顺序来执行规则。

常见的路由模式是提供一或多个高优先级规则，这些优先规则使用源服务以及 Header 来进行路由判断，然后才提供一条单独的基于权重的规则，这些低优先级规则不设置匹配规则，仅根据权重对所有剩余流量进行分流。

目标规则

在请求被 VirtualService 路由之后，
DestinationRule 配置的一系列策略就生效了。
这些策略由服务属主编写，包含断路器、负载均衡以及 TLS 等的配置内容。

```
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews
  trafficPolicy:
    loadBalancer:
      simple: RANDOM
  subsets:
  - name: v1
    labels:
      version: v1
  - name: v2
    labels:
      version: v2
  trafficPolicy:
    loadBalancer:
      simple: ROUND_ROBIN
  - name: v3
    labels:
      version: v3
```


断路器

可以用一系列的标准，例如连接数和请求数限制来定义简单的断路器。

可以通过定义 outlierDetection 自定义健康检查模式。

```
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: httpbin
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 1
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
    outlierDetection:
      consecutiveErrors: 1
      interval: 1s
      baseEjectionTime: 3m
      maxEjectionPercent: 100
```

ServiceEntry

Istio 内部会维护一个服务注册表，可以用 `ServiceEntry` 向其中加入额外的条目。通常这个对象用来启用对 Istio 服务网格之外的服务发出请求。

`ServiceEntry` 中使用 `hosts` 字段来指定目标，字段值可以是一个完全限定名，也可以是个通配符域名。其中包含的白名单，包含一或多个允许网格中服务访问的服务。

只要 `ServiceEntry` 涉及到了匹配 `host` 的服务，就可以和 `VirtualService` 以及 `DestinationRule` 配合工作。

```
apiVersion: networking.istio.io/v1alpha3
kind: ServiceEntry
metadata:
  name: foo-ext-svc
spec:
  hosts:
  - *.foo.com
  ports:
  - number: 80
    name: http
    protocol: HTTP
  - number: 443
    name: https
    protocol: HTTPS
```

WorkloadEntry

```
apiVersion: networking.istio.io/v1beta1
kind: ServiceEntry
metadata:
  name: details-svc
spec:
  hosts:
    - details.bookinfo.com
  location: MESH_INTERNAL
  ports:
    - number: 80
      name: http
      protocol: HTTP
  resolution: STATIC
  workloadSelector:
    labels:
      app: details-legacy
```

```
apiVersion: networking.istio.io/v1beta1
kind: WorkloadEntry
metadata:
  name: details-svc
spec:
  serviceAccount: details-legacy
  address: 2.2.2.2
  labels:
    app: details-legacy
    instance-id: vm1
```

Gateway

Gateway 为 HTTP/TCP 流量配置了一个负载均衡，多数情况下在网格边缘进行操作，用于启用一个服务的入口（ingress）流量。

和 Kubernetes Ingress 不同，Istio Gateway 只配置四层到六层的功能（例如开放端口或者 TLS 配置）。绑定一个 VirtualService 到 Gateway 上，用户就可以使用标准的 Istio 规则来控制进入的 HTTP 和 TCP 流量。

```
apiVersion:
networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: bookinfo-gateway
spec:
  servers:
  - port:
      number: 443
      name: https
      protocol: HTTPS
    hosts:
    - bookinfo.com
  tls:
    mode: SIMPLE
    serverCertificate: /tmp/tls.crt
    privateKey: /tmp/tls.key
```

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: bookinfo
spec:
  hosts:
  - bookinfo.com
  gateways:
  - bookinfo-gateway
  http:
  - match:
    - uri:
        prefix: /reviews
    route:
    ...
```

开启网关安全加固

```
apiVersion:
networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: bookinfo-gateway
spec:
  servers:
  - port:
      number: 443
      name: https
      protocol: HTTPS
    hosts:
    - bookinfo.com
  tls:
    mode: SIMPLE
    serverCertificate: /tmp/tls.crt
    privateKey: /tmp/tls.key
```

```
apiVersion:
networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: bookinfo-gateway
spec:
  servers:
  - port:
      number: 443
      name: https
      protocol: HTTPS
    hosts:
    - bookinfo.com
  tls:
    mode: SIMPLE
    credentialName: foo
```

遥测 (Telemetry V2)

基本 Metrics

针对 HTTP, HTTP/2 和 GRPC 协议, Istio 收集以下指标:

- Request Count (istio_requests_total)
 - 请求总数
- Request Duration (istio_request_duration_milliseconds)
 - 请求处理时长
- Request Size (istio_request_bytes)
 - 请求包大小
- Response Size (istio_response_bytes)
 - 响应包大小

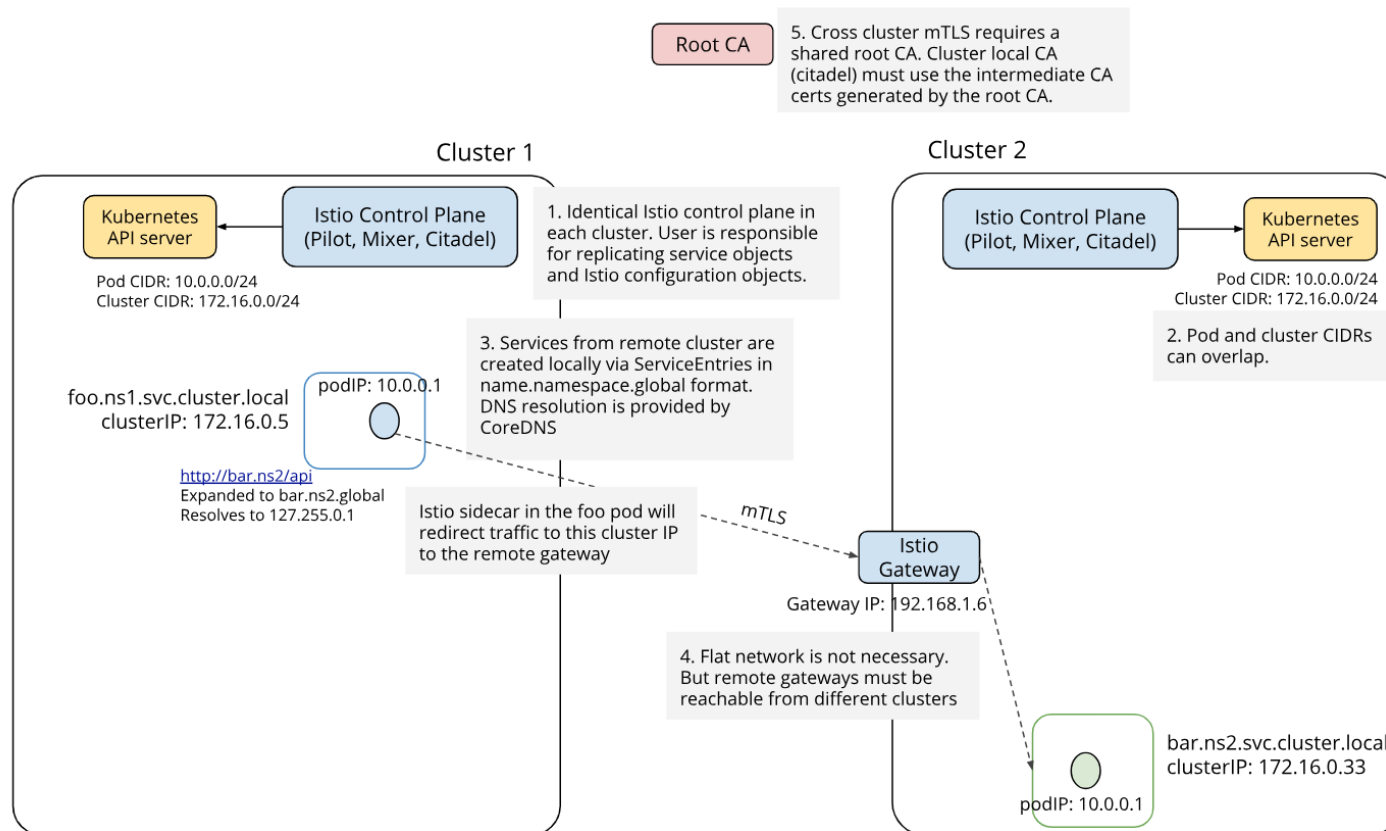
针对 TCP 协议, Istio 收集以下指标:

- Tcp Byte Sent (istio_tcp_sent_bytes_total)
 - 发送的数据响应包的总大小
- Tcp Byte Received (istio_tcp_received_bytes_total)
 - 接收到的数据包大小
- Tcp Connections Opened (istio_tcp_connections_opened_total)
 - TCP 连接数
- Tcp Connections Closed (istio_tcp_connections_closed_total)
 - 关闭的 TCP 连接数
- 同时支持 WASM (Web Assembly) plugin 收集数据
 - 但启用 WASM 后资源开销显著上升

多集群

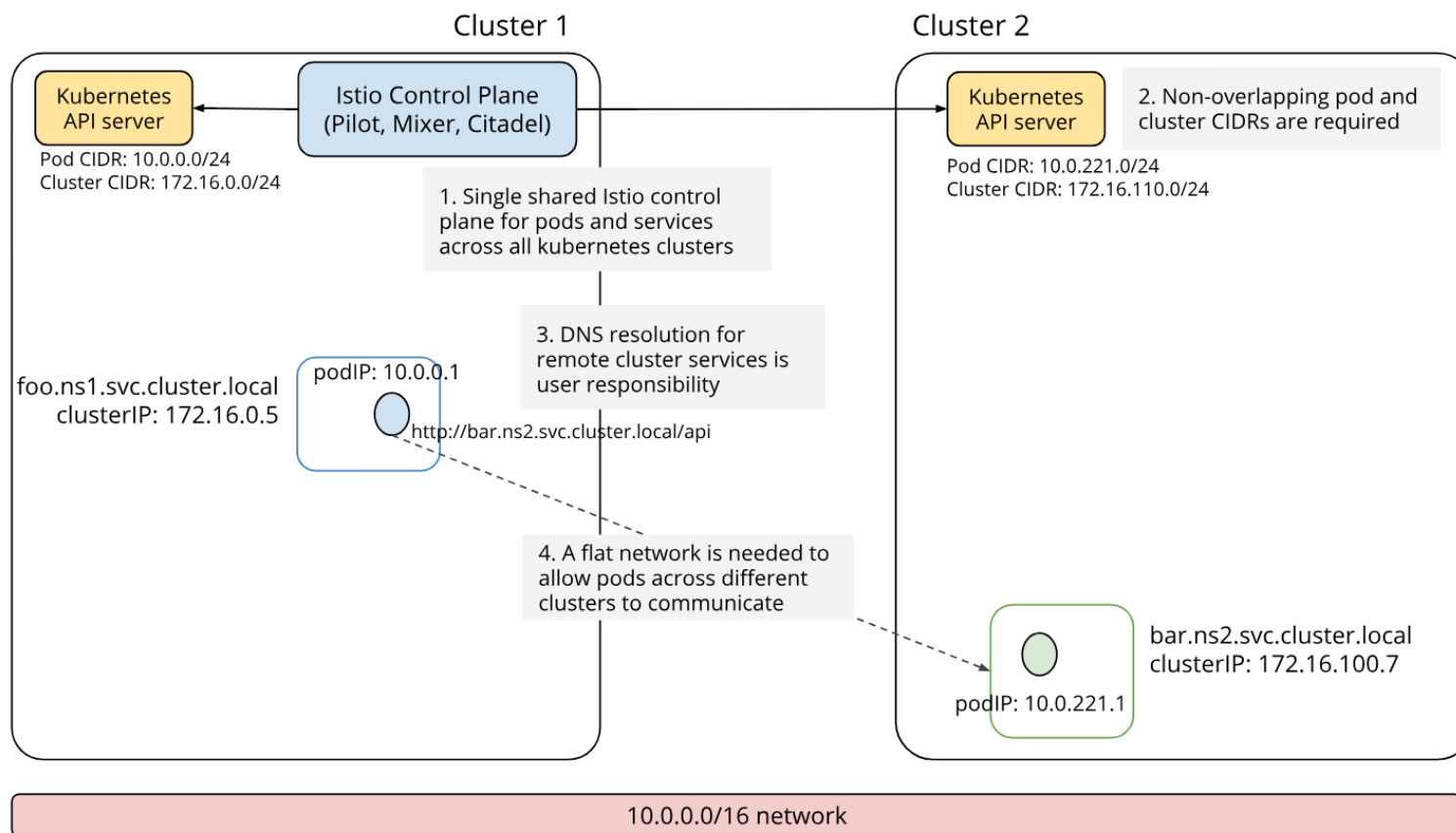
多集群网格扩展

基于 Gateway



多集群网格扩展

基于 VPN 或可互通的集群



5. 跟踪采样

跟踪采样配置

Istio 默认捕获所有请求的跟踪。例如，何时每次访问时都使用上面的 Bookinfo 示例应用程序 / productpage 你在 Jaeger 看到了相应的痕迹仪表板。此采样率适用于测试或低流量目。

- 在运行的网格中，编辑 istio-pilot 部署并使用以下步骤更改环境变量：

```
istioctl upgrade --set values.global.tracer.zipkin.address=jaeger-collector:9411
```

gw

svc0

svc1

svc2

应用程序埋点

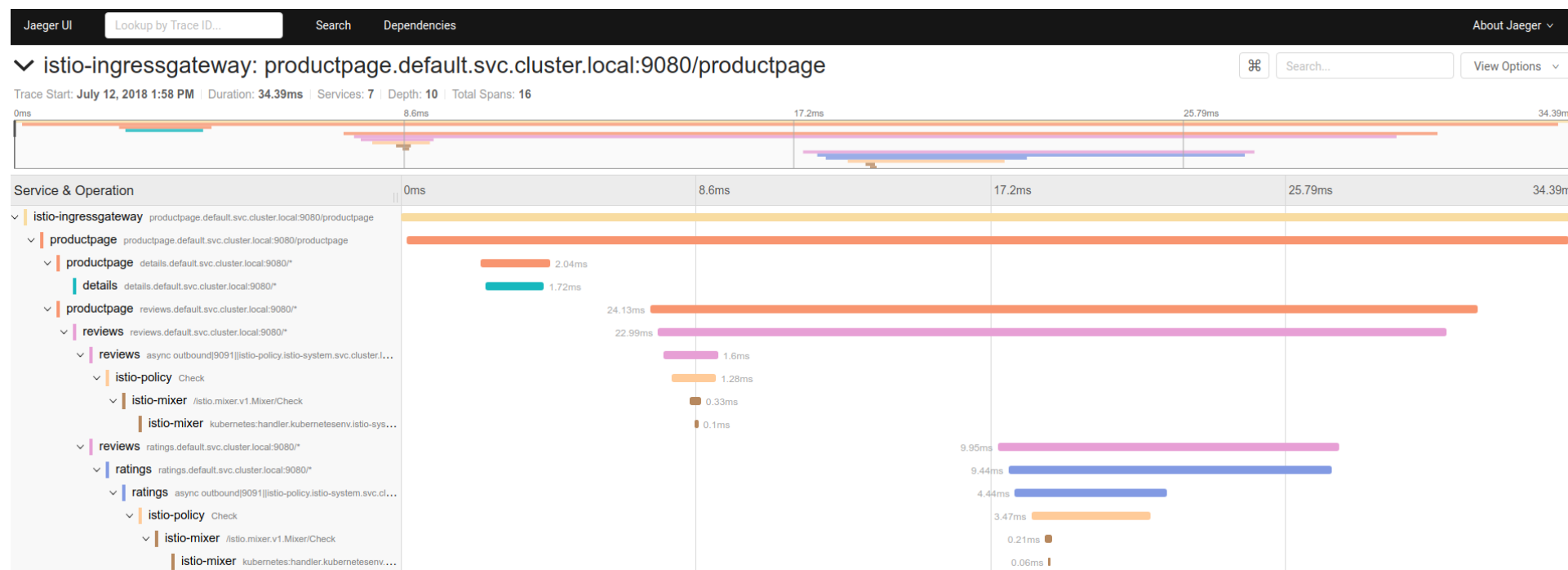
虽然 Istio 代理能够自动发送 Span 信息，但还是需要一些辅助手段来把整个跟踪过程统一起来。应用程序应该自行传播跟踪相关的 HTTP Header，这样在代理发送 Span 信息的时候，才能正确的把同一个跟踪过程统一起来。

为了完成跟踪的传播过程，应用应该从请求源头中收集下列的 HTTP Header，并传播给外发请求：

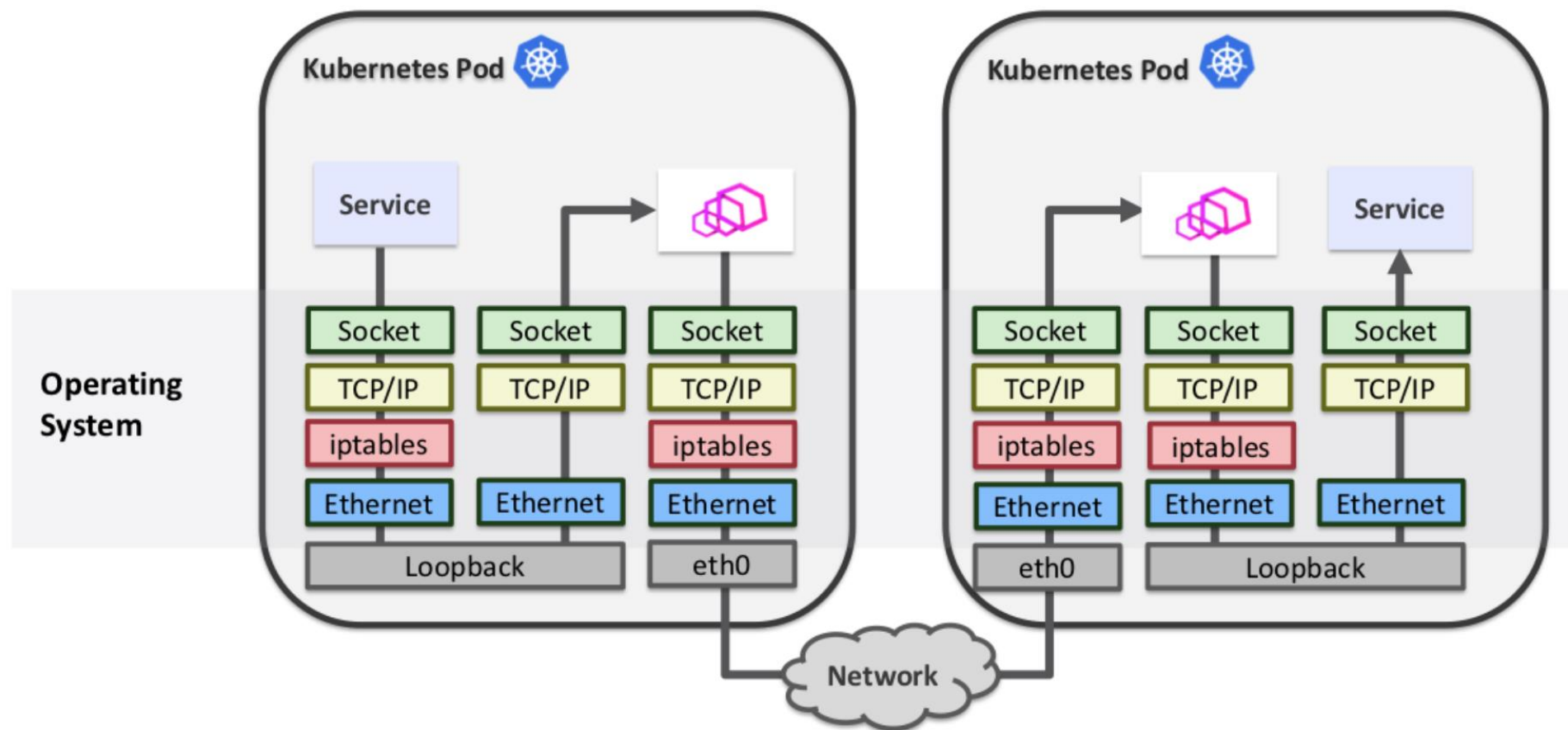
- x-request-id
- x-b3-traceid
- x-b3-spanid
- x-b3-parentspanid
- x-b3-sampled
- x-b3-flags
- x-ot-span-context

分布式跟踪

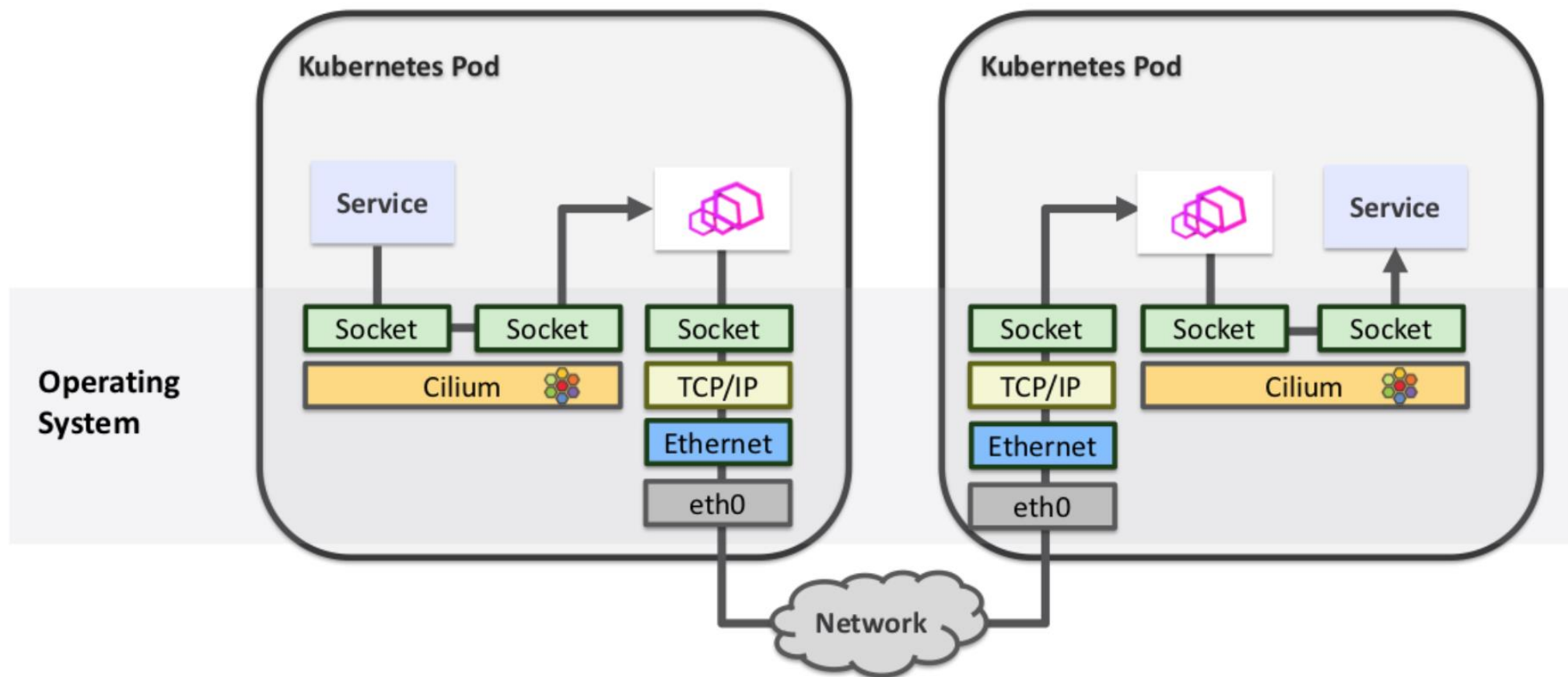
跟踪信息详细视图



Service Mesh 涉及的网络栈



Cilium 数据平面加速



小结

微服务架构是当前业界普遍认可的架构模式，容器的封装性，隔离性为微服务架构的兴盛提供了有力的保障。

Kubernetes 作为声明式集群管理系统，代表了分布式系统架构的大方向：

- kube-proxy 本身提供了基于 iptables/ipvs 的四层 Service Mesh 方案；
- Istio/linkerd 作为基于 Kubernetes 的七层 Service Mesh 方案，近期会有比较多的生产部署案例。

生产系统需要考虑的，除了 Service Mesh 架构带来的便利性，还需要考虑：

- 配置一致性检查；
- endpoint 健康检查；
- 海量转发规则情况下的 scalability。

作业

把我们的 httpserver 服务以 Istio Ingress Gateway 的形式发布出来。以下是你需要考虑的几点：

- 如何实现安全保证；
- 七层路由规则；
- 考虑 open tracing 的接入。

THANKS

 极客时间 | 训练营